Final Project Report

K-POP COMEBACK SENTIMENT ANALYSIS

Investigating the Correlation Between Social Media Sentiment and YouTube Performance Metrics

Miracle W. Okoi-Obuli Student Number: 24129002 Academic Supervisor: Lili Kirner

April 2026

UNIVERSITY OF HERTFORDSHIRE
Department of Computer Science
BSc Honours Computer Science (Online)
6WCM0029-0105-2025 - Computer Science Project (COM)

Acknowledgements

I would like to thank my supervisor, Lili Kirner, for her guidance and feedback throughout this project. I am also grateful to my family and friends for their support during my final year.

Abstract

This project investigates whether social media sentiment expressed by K-pop fans correlates with the YouTube performance of comeback music videos. Six comebacks from 2024 to 2025 were selected as case studies: aespa Whiplash, IVE Rebel Heart, TWICE Strategy, NCT DREAM When I'm With You, ATEEZ Ice On My Teeth, and Stray Kids Chk Chk Boom.

Reddit comments were collected from Pushshift monthly archive dumps, and YouTube comments were collected using the YouTube Data API v3. Text data was preprocessed using tokenisation, stopword removal, and Porter stemming. A combined dataset of 597 manually labelled comments was used to train Naïve Bayes and Support Vector Machine classifiers in WEKA using 10-fold cross-validation. SVM achieved 65.49% accuracy and was selected as the final model.

The classifier was applied to a full corpus of 16,952 comments to generate sentiment distributions for each comeback. Pearson and Spearman correlation coefficients were computed between sentiment scores and four YouTube engagement metrics. No statistically significant correlation was found between YouTube sentiment and any engagement metric. A negative correlation was observed between Reddit sentiment and like-to-view ratio (Pearson $r = -0.795$, $p = 0.059$), although this did not reach conventional significance.

The findings indicate that the relationship between fan sentiment and digital performance is not directly linear and is likely influenced by factors such as fanbase size and engagement behaviour. Results are presented in an interactive Tableau dashboard.

Contents

Chapter 1: Introduction

Project Aim

This project investigates whether social media sentiment expressed by K-pop fans correlates with the YouTube performance of comeback music videos. Six comebacks from 2024–2025 were selected as case studies, with sentiment data collected from Reddit and YouTube comments and performance metrics collected via the YouTube Data API v3.

Core Objectives

C1. To collect Reddit comments referencing each of six K-pop comebacks from nine relevant subreddits using the Pushshift archive dumps, within a 14-day window from each comeback's official release date.

C2. To collect YouTube engagement metrics, including view count, like count, comment count, and like-to-view ratio, for the official music video of each of the six comebacks using the YouTube Data API v3.

C3. To preprocess the collected text to produce a clean corpus suitable for classification, applying tokenisation, lowercasing, URL and emoji handling, stopword removal, and feature extraction.

C4. To manually label a stratified sample of 50 YouTube comments per comeback, resulting in 300 comments in total, as positive, negative, or neutral, and to export the labelled dataset in ARFF format for use in WEKA.

C5. To train and evaluate Naïve Bayes and Support Vector Machine classifiers in WEKA using 10-fold cross-validation, and to report accuracy, precision, recall, and F1-score for each algorithm.

C6. To apply the best-performing classifier to the full preprocessed corpus, including both YouTube and Reddit data, compute aggregated sentiment distributions for each comeback, and report held-out validation accuracy on a labelled Reddit sample to assess domain transfer performance.

C7. To test for statistical correlation between aggregated sentiment and YouTube engagement metrics across the six comebacks using Pearson and Spearman coefficients, and to report both the coefficient and associated p-value.

C8. To produce an interactive Tableau dashboard presenting sentiment distributions, YouTube engagement metrics, and correlation results across the six comebacks.

All core objectives were planned for completion by the project submission deadline of 26 April 2026.

Advanced Objectives

Three advanced objectives proposed in the Interim Progress Report, namely comparison of Reddit and Twitter sentiment (A1), temporal sentiment analysis across the 14-day window (A2), and comparison of sentiment patterns between girl groups and boy groups (A3), were not pursued. A1 was not feasible due to lack of access to the Twitter API, which was identified as a constraint during the Interim Report phase. A2 and A3 were deferred due to time constraints following adjustments to the core methodology. These limitations are discussed further in the Conclusion.

Glossary

Term	Definition

Comeback	A K-pop group's release of new music, typically accompanied by a music video, promotional activities, and fan engagement.
MV	Music Video.
Fandom	The collective community of fans of a particular K-pop group.
Sentiment analysis	The computational identification of subjective opinion expressed in text.
WEKA	Waikato Environment for Knowledge Analysis — a machine learning toolkit.
SVM	Support Vector Machine — a supervised classification algorithm.
Naïve Bayes	A probabilistic supervised classification algorithm.
TF-IDF	Term Frequency-Inverse Document Frequency — a text feature extraction method.
Pushshift	A publicly available archive of Reddit data used in academic research.
Subreddit	A community forum on Reddit dedicated to a specific topic.
Pearson r	A measure of linear correlation between two continuous variables.
Spearman ρ	A rank-based measure of monotonic correlation.

Ethics Statement

This project did not require ethics approval from the University of Hertfordshire. I analysed publicly available secondary data and did not involve human participants, interviews, surveys, or experiments. I obtained Reddit comments from the Pushshift monthly archive dumps, a publicly available dataset widely used in academic research. I collected YouTube engagement metrics via the YouTube Data API v3.

I did not collect, store, or report any personally identifying information. I present all findings at the level of aggregate sentiment distributions per comeback, not at the level of individual users or comments. I discussed the need for ethics approval with my project supervisor.

AI Use Statement

I developed this report independently. I used Claude (Anthropic, claude.ai) in a limited capacity during the development phase to support debugging of Python scripts, clarify technical concepts, and assist in resolving development issues. I also used Grammarly (an AI-powered writing assistant) for proofreading purposes, including identifying grammatical errors, improving sentence clarity, and suggesting minor language corrections.

During the literature search phase, I used Google Scholar Labs, an AI-assisted feature within Google Scholar, to identify relevant academic sources. The tool was used to explore research topics and locate candidate papers. All suggested sources were independently verified using Google Scholar, and only papers that I manually reviewed by reading their abstracts, introductions, and conclusions were included.

I did not use any AI tools to generate or write the final report content, develop the methodology, perform the analysis, or produce code presented as my own without review. All ideas, decisions, analyses, and technical work presented are my own.

All suggestions provided by AI tools were critically reviewed and manually implemented where appropriate, in accordance with the University of Hertfordshire AI policy.

Chapter Summaries

Literature Review reviews existing research in sentiment analysis, social media analytics applied to entertainment performance, and K-pop as a research subject, identifying the gap addressed by this project.

Requirements sets out the functional and non-functional requirements derived from the project objectives, including legal, ethical, professional, and environmental constraints.

Design describes the architecture of the end-to-end sentiment analysis pipeline, from data collection through to visualisation.

Technical Development details the implementation of each pipeline stage, including data collection, preprocessing, manual labelling, classification, correlation analysis, and dashboard construction, alongside challenges encountered and decisions made.

Evaluation reports the performance of the Naïve Bayes and Support Vector Machine classifiers using 10-fold cross-validation metrics.

Results presents the sentiment distributions per comeback, YouTube engagement metrics, and correlation coefficients between sentiment and performance.

Discussion analyses the key findings, considers their implications, and acknowledges the limitations of the study.

Conclusion reflects on the extent to which each project objective was met and proposes directions for future work.

Chapter 2: Literature Review

2.1 Search Strategy

Literature was identified through a structured multi-stage discovery process. Initial source identification was conducted using Google Scholar Labs, an AI-assisted search feature within Google Scholar, to explore research topics and generate a broad set of potentially relevant academic publications. These suggestions were treated as starting points rather than authoritative sources.

All candidate sources were then manually searched and verified using Google Scholar to confirm their authenticity, accessibility, and academic relevance. Each paper was independently reviewed by reading its abstract, introduction, and conclusion to assess its suitability for inclusion.

Only sources that met these criteria were selected for use. Three sources were chosen for detailed summary in the Interim Progress Report, with additional relevant sources incorporated into the Final Project Report bibliography. All summaries and discussions of the literature were written independently.

2.2 Sentiment Analysis

Sentiment analysis is a subfield of natural language processing (NLP) concerned with identifying and classifying subjective opinions expressed in text, typically as positive, negative, or neutral (Pang and Lee, 2008; Liu, 2012). It is widely applied to user-generated content, where large volumes of unstructured data can be analysed computationally to mine public opinion.

Two main approaches are commonly used: lexicon-based methods and machine learning-based methods. Lexicon-based approaches rely on predefined sentiment dictionaries, such as VADER or SentiWordNet, where words are assigned polarity scores. While simple to implement, they often struggle with contextual nuance and domain-specific language (Liu, 2012). In contrast, machine learning approaches involve training classifiers on labelled datasets, allowing models to learn patterns directly from the data (Pang and Lee, 2008).

Among machine learning techniques, Naïve Bayes and Support Vector Machines (SVM) are widely used for text classification. Comparative studies have shown that both algorithms perform effectively in sentiment classification tasks, including multilingual and social media contexts (Aprinastya *et al.*, 2024; Millennianita, Athiyah and Muhammad, 2024). These models remain the industry standard for supervised classification due to their reliability and interpretability.

2.3 Social Media Sentiment and Entertainment Performance

A growing body of research has explored the relationship between social media sentiment and real-world performance metrics. Early work by Asur and Huberman (2010) demonstrated that Twitter activity could be used to predict box office revenue, highlighting social media's potential as a real-time indicator of public interest.

Subsequent studies have extended this into the music domain. Rui, Liu and Whinston (2013) found that both the volume and valence of online discussions influence product sales, while Tsiara and Tjortjis (2020) showed that Twitter data can be used to predict chart positions for songs. More recent research suggests that sentiment does not act in isolation; Reisz, Servedio and Thurner (2024) found that

homophily and influencer networks play a significant role in shaping song popularity. Across these studies, a consistent finding is that sentiment expressed during early-release periods shows measurable correlations with performance metrics.

2.4 K-pop as a Research Subject

K-pop has attracted increasing academic attention due to its global reach and highly organised fan communities. Researchers have examined fan behaviour, including coordinated streaming practices and collective actions aimed at boosting chart performance (Jung, 2011).

The comeback cycle is a defining feature of the industry, where artists release new music accompanied by structured promotional campaigns. These cycles create predictable periods of heightened activity across social media, making them particularly suitable for sentiment analysis. Recent research by Zhang et al. (2025) highlights the intensity and immediacy of fan responses in these online environments. Despite this, much of the existing literature focuses on sociocultural aspects or specific regional audiences. There is a notable lack of research investigating the quantitative relationship between English-language fan sentiment and YouTube engagement metrics.

2.5 Summary and Research Gap

Sentiment analysis is a well-established field, and prior research has demonstrated links between social media sentiment and performance across domains such as film and music. However, K-pop research currently prioritises cultural analysis over quantitative relationships between sentiment and platform-specific metrics. This project addresses that gap by combining sentiment classification with YouTube performance data to determine how English-language fan sentiment relates to the commercial success of K-pop comebacks.

Chapter 3: Requirements

3.1 Requirements Capture Methodology

The requirements for this project were derived directly from the project aim and core objectives to ensure that the system design remained aligned with the intended research outcome. Rather than defining features independently, each requirement was formulated to support a specific stage of the sentiment analysis pipeline, from data collection through to correlation analysis.

A goal-oriented approach was used to structure the requirements. This allowed each requirement to be mapped explicitly to one or more objectives, ensuring traceability and making it possible to evaluate whether the system met its intended purpose. Requirements were also written in a measurable form so that their completion could be verified through implementation and testing.

Both functional and non-functional requirements were identified. Functional requirements describe the core capabilities required to carry out the analysis, while non-functional requirements define constraints relating to reproducibility, consistency, and ethical handling of data.

The MoSCoW method was applied to prioritise requirements. Given the limited project timeframe, all core analytical components were classified as Must requirements, as each one is necessary to produce meaningful results. This reflects the dependency structure of the pipeline, where omission of any stage would prevent the project from meeting its aim.

3.2 Functional Requirements

TABLE 3.2 - FUNCTIONAL REQUIREMENTS

ID	Requirement	Priority	Objective
FR1	To collect Reddit comments from nine specified subreddits for each of six K-pop comebacks within a 14-day window from release date	Must	C1
FR2	To collect YouTube engagement metrics (view count, like count, comment count, like-to-view ratio) for each comeback's official MV	Must	C2
FR3	To preprocess all collected text using tokenisation, lowercasing, URL and emoji handling, stopword removal, and feature extraction	Must	C3
FR4	To manually label a stratified sample of 50 YouTube comments per comeback as positive, negative, or neutral	Must	C4
FR5	To export the labelled dataset in ARFF format for use in WEKA	Must	C4
FR6	To train and evaluate Naïve Bayes and Support Vector Machine classifiers using 10-fold cross-validation	Must	C5
FR7	To report accuracy, precision, recall, and F1-score for both classifiers	Must	C5
FR8	To apply the best-performing classifier to the full corpus and generate sentiment distributions per comeback	Must	C6
FR9	To compute Pearson and Spearman correlation coefficients between sentiment and YouTube engagement metrics	Must	C7
FR10	To produce an interactive Tableau dashboard visualising sentiment and engagement relationships	Must	C8

3.3 Non-Functional Requirements

TABLE 3.3 - NON-FUNCTIONAL REQUIREMENTS

ID	Requirement	Priority
NFR1	All data collection processes must be reproducible using documented Pushshift archive datasets	Must
NFR2	All code must be version-controlled using Git and stored in a GitHub repository to support traceability	Must
NFR3	No personally identifiable information must be collected, stored, or reported at any stage of the project	Must
NFR4	The preprocessing pipeline must be applied consistently across Reddit and YouTube datasets to ensure comparability	Must
NFR5	The final Tableau dashboard should be publicly accessible via Tableau Public to allow external review of results	Should

3.4 Legal, Ethical, Professional, and Environmental Constraints

The constraints affecting this project were considered during the requirements definition stage to ensure that all design decisions remained compliant with academic and professional standards.

From a legal perspective, all Reddit data was sourced from the Pushshift archive, which provides publicly available historical Reddit data and is widely used in academic research. YouTube data was collected using the official YouTube Data API v3 in accordance with its terms of service. The project does not reproduce copyrighted media content and uses all data solely for academic analysis.

Ethically, the project does not involve human participants, surveys, or interviews. All data analysed is publicly available, and no personally identifiable information is collected or retained. The analysis is conducted at an aggregate level, focusing on overall sentiment distributions rather than individual users. Based on these factors, ethics approval was not required, which was confirmed through discussion with the project supervisor.

From a professional standpoint, the project aligns with the **BCS CODE OF CONDUCT** (BCS, 2022), particularly in relation to integrity, responsible data handling, and maintaining transparency in methodology. The use of AI tools has been fully declared and limited to acceptable use as defined by University policy.

Environmentally, all processing was conducted on a personal machine without reliance on large-scale computational resources. The use of existing datasets avoids unnecessary data collection from live systems, reducing additional load on external platforms.

Chapter 4: Design

4.1 System Architecture

The system was designed as a sequential pipeline that transforms raw social media data into measurable relationships between sentiment and performance. This pipeline structure was chosen because the project involves multiple dependent stages, where each stage produces the input required for the next.

The first stage involves data collection. YouTube engagement metrics are retrieved using the YouTube Data API v3, while Reddit comments are extracted from the Pushshift archive. These sources were selected because they provide structured access to large volumes of publicly available data and allow consistent retrieval across all selected comebacks.

The second stage applies filtering to improve data relevance. Keyword-based filtering is used for Reddit comments to remove unrelated discussions, while language filtering ensures that only English-language content is retained. This decision was made to maintain consistency in the sentiment analysis process, as the classification models are trained on English-language data.

The preprocessing stage standardises the text data across both platforms. This includes lowercasing, URL removal, tokenisation, stopword removal, and stemming. A consistent preprocessing pipeline was applied to both datasets to ensure that any differences observed in sentiment are not due to inconsistencies in text handling.

The labelling stage introduces supervised learning into the system. A manually labelled dataset was created using a stratified sample of YouTube comments. This approach was selected to ensure that each comeback is equally represented in the training data, reducing bias in the classification model.

In the classification stage, Naïve Bayes and Support Vector Machine models are trained and evaluated using WEKA. These models were selected based on their established performance in text classification tasks and their suitability for relatively small, labelled datasets. Ten-fold cross-validation was used to evaluate model performance, providing a reliable estimate of generalisation accuracy.

The final stage involves analysis and visualisation. Sentiment predictions are aggregated per comeback and compared with YouTube engagement metrics using Pearson and Spearman correlation coefficients. These measures were selected to capture both linear and rank-based relationships. Results are presented through an interactive Tableau dashboard to support clear interpretation.

FIGURE 4.1: END-TO-END SENTIMENT ANALYSIS PIPELINE

4.2 Tool Selection

The tools used in this project were selected based on their suitability for each stage of the pipeline and their alignment with standard practices in data science and machine learning.

TABLE 4.2 - TOOL SELECTION

Tool	Purpose	Justification
Python	Data processing and scripting	Provides flexibility for handling large datasets and integrating multiple stages of the pipeline
NLTK	Text preprocessing	Supports tokenisation, stopword removal, and stemming required for text normalisation
scikit-learn	Feature extraction and modelling	Used for TF-IDF vectorisation and model application to the full dataset
WEKA	Model training and evaluation	Selected to meet project requirements for structured classifier evaluation
Tableau Public	Data visualisation	Enables interactive exploration of sentiment and performance relationships
GitHub	Version control	Supports reproducibility and transparent development tracking
Pushshift	Reddit data source	Provides historical Reddit data suitable for retrospective analysis
YouTube Data API v3	YouTube metrics	Official and reliable method for retrieving engagement data

Tool selection reflects a balance between academic requirements, practical implementation, and reproducibility.

4.3 Data Flow Overview

The system follows a structured data flow from raw data collection to final analysis.

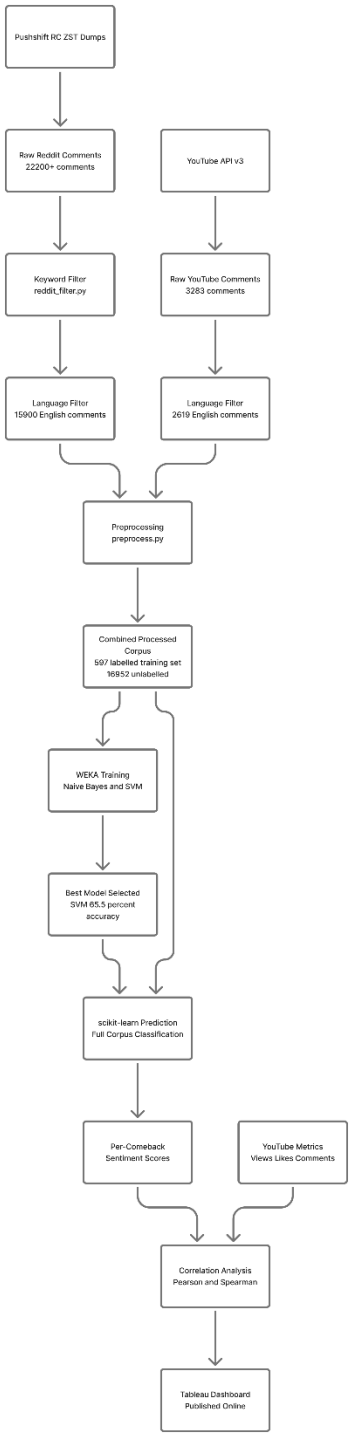


FIGURE 4.2: DATA FLOW DIAGRAM

Data is first collected from Reddit and YouTube, then filtered to remove irrelevant or non-English content. The filtered data is preprocessed into a consistent format suitable for machine learning. A labelled subset is then created and used to train classification models.

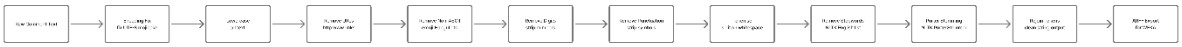


FIGURE 4.3: TEXT PREPROCESSING PIPELINE

The trained model is applied to the full dataset to generate sentiment predictions. These predictions are aggregated per comeback and combined with YouTube engagement metrics. Statistical analysis is then performed to identify relationships between sentiment and performance, and the results are visualised.

This sequential flow ensures that each stage contributes directly to the overall objective of the project.

4.4 Labelling Codebook Design

A structured labelling codebook was developed prior to annotation to ensure consistency in sentiment classification. Three classes were defined: positive, negative, and neutral.

Positive comments indicate approval or enjoyment of the comeback, while negative comments reflect criticism or dissatisfaction. Neutral comments include factual statements or content without a clear sentiment.

To reduce ambiguity, specific rules were defined for handling edge cases such as sarcasm, mixed sentiment, and fandom-specific language. In these cases, sentiment was assigned based on the dominant meaning directed toward the comeback rather than surface-level wording.

The use of a predefined codebook was necessary to maintain consistency across the labelled dataset and improve the reliability of the classification models.

Chapter 5: Technical Development

5.1 Development Environment

This project was developed using Python 3.13.9 within the Anaconda distribution on a Windows 11 laptop. Initial data collection and exploratory work were carried out in Jupyter Notebook, which allowed for iterative testing and debugging. As the project progressed, scripts were refactored into standalone Python files stored in the 10_scripts/ directory to improve structure and reproducibility.

All scripts were configured using absolute file paths specific to the development environment. These would require modification for execution on a different system.

Version control was managed using Git, with the repository hosted on GitHub at: https://github.com/mimiokoiobuli/kpop-sentiment-analysis

Commit messages were used to document each stage of development, providing a clear record of how the project evolved over time. This approach supports reproducibility and aligns with the non-functional requirement for traceability.

5.2 Data Collection

5.2.1 YouTube Engagement Metrics

YouTube engagement metrics were collected using the YouTube Data API v3 via the google-api-python-client library. For each of the six comeback music videos, view count, like count, and comment count were retrieved. A like-to-view ratio was also calculated as a derived metric to provide a normalised measure of engagement.

The API returns cumulative metrics rather than time-window-specific values. As a result, all metrics represent totals at the time of collection in March 2026. This limitation affects the ability to directly align engagement metrics with the 14-day sentiment window and is considered in the Discussion chapter.

The API key was stored in a local credentials file excluded from version control using .gitignore. During development, a Jupyter checkpoint file containing a hardcoded API key was unintentionally committed. After receiving an automated alert, the key was immediately revoked and replaced with a restricted key. The exposed file was removed from the repository history using git filter-branch, and the cleaned history was force-pushed. This incident highlighted the importance of excluding temporary notebook files from version control and reinforced secure key management practices.

5.2.2 YouTube Comments

YouTube comments were collected using the commentThreads().list() endpoint of the YouTube Data API v3. Comments were filtered to those posted within a 14-day window from each comeback's release date to align with the defined analysis period.

An initial collection attempt used relevance-based sorting, which returned a large number of recent comments rather than those from the comeback period. This was corrected by switching to date-based sorting and applying explicit date filtering. Approximately 500 comments per comeback were collected, resulting in a total of 3,283 comments across all six case studies.

5.2.3 Reddit Comments

Reddit comments were collected from Pushshift monthly archive dumps rather than the Reddit API, which was not approved in time. The Pushshift dataset consists of compressed newline-delimited JSON files (RC_*.zst) containing all comments for a given month. Five monthly dumps were processed, covering July 2024 to January 2025.

For each comeback, comments were collected from nine subreddits: three general K-pop subreddits and six group-specific subreddits. Comments were retained if they fell within the defined 14-day window. Comments from group-specific subreddits were included by default, while comments from general subreddits were required to contain the group name or song title.

During development, an inconsistency was identified in the IVE dataset. The initial collection captured discussion related to a different release than the one used for YouTube metrics. This was corrected by reprocessing the relevant monthly archive using the correct release date window. This step ensured consistency between sentiment data and performance metrics across all case studies.

The original collection script was lost prior to being committed to version control. It was reconstructed by analysing the structure and content of the existing dataset and re-implementing the filtering logic. While not ideal, this reconstruction ensured that the data collection process remained consistent with the intended methodology.

5.3 Data Filtering

5.3.1 Keyword Filter Refinement

Initial keyword filtering revealed a high level of noise for group names that overlap with common English words. For example, the substring “ive” matched contractions such as “I’ve” and words such as “give,” resulting in a large proportion of irrelevant matches. Similar issues were observed for “twice” and for song titles that overlap with common phrases.

To address this, a refined filtering approach was implemented using regular expressions with word boundaries. Case-sensitive patterns were used for ambiguous group names, and additional conditions were applied for ambiguous song titles, requiring co-occurrence with the group name or member names.

This refinement significantly reduced noise while preserving relevant content. Groups with distinctive names showed minimal change, indicating that the filtering primarily removed false positives rather than valid data. This step was important to ensure that the sentiment analysis was based on relevant discussion.

5.3.2 Language Filter

To maintain consistency with the project methodology, only English-language content was included. A two-stage filtering approach was implemented.

First, comments containing substantial non-Latin script characters were removed. Second, for longer comments, the langdetect library was used with a confidence threshold to identify English text. Short comments were retained due to the unreliability of language detection on very short inputs.

Reddit data required minimal filtering, while YouTube data showed a higher proportion of non-English content, particularly for groups with international audiences. This difference reflects platform-specific audience composition and was accounted for in the analysis.

5.4 Preprocessing

A consistent preprocessing pipeline was implemented to standardise text data across both Reddit and YouTube sources. The pipeline included encoding normalisation, lowercasing, URL removal, non-ASCII character handling, digit and punctuation removal, tokenisation, stopword removal, and stemming.

Although a partially cleaned version of the YouTube dataset already existed, it was reprocessed using the full pipeline to ensure consistency with the stated methodology. Applying the same preprocessing steps to both datasets was necessary to ensure that the classifier operated on comparable feature representations.

The processed data was exported as CSV files and converted into ARFF format for use in WEKA. During this process, a naming conflict was identified with the class attribute. This was resolved by adopting the naming convention required by WEKA's filtering pipeline, ensuring compatibility during model training.

5.5 Manual Labelling

A total of 597 comments were manually labelled as positive, negative, or neutral. YouTube comments were selected sequentially to reflect typical user engagement, while Reddit comments were sampled randomly with a fixed seed to ensure reproducibility.

A labelling codebook was developed prior to annotation to define each sentiment class and provide rules for handling edge cases such as sarcasm and mixed sentiment. This helped maintain consistency during labelling and reduced subjectivity.

The resulting dataset showed a strong class imbalance, with a low proportion of negative comments. This reflects the nature of fan discourse rather than an issue with the labelling process. However, it introduced challenges for classification, particularly in detecting negative sentiment.

5.6 Classification

Two classifiers, Naïve Bayes and Support Vector Machine (SVM), were trained and evaluated using WEKA with 10-fold cross-validation. Text data was vectorised using TF-IDF weighting.

SVM achieved higher accuracy and F1-score compared to Naïve Bayes and was selected as the final model. Both classifiers showed limited performance on the negative class, largely due to class imbalance in the training data.

For full dataset prediction, a technical limitation was encountered with WEKA's export functionality. To address this, the classification pipeline was reimplemented in scikit-learn using equivalent parameters. The results were consistent with those obtained in WEKA, providing confidence in the implementation.

The trained model was then applied to the full Reddit and YouTube datasets to generate sentiment predictions for each comeback.

5.7 Correlation Analysis

Sentiment scores were calculated for each comeback using the difference between positive and negative proportions. These scores were compared with YouTube engagement metrics using Pearson and Spearman correlation coefficients.

Both correlation measures were used to capture different types of relationships, with Pearson measuring linear correlation and Spearman measuring rank-based relationships. The analysis was implemented using the scipy.stats library, and both coefficients and p-values were reported.

5.8 Tableau Dashboard

An interactive dashboard was developed in Tableau Public to present the results of the analysis. The dashboard includes visualisations of engagement metrics, sentiment distributions, and correlation relationships.

The use of an interactive dashboard allows results to be explored dynamically and supports clearer interpretation of patterns across comebacks. The dashboard is publicly accessible at: https://public.tableau.com/views/KPop_Sentiment_Analysis_Dashboard/K-popSentimentDashboard

The implementation follows a structured end-to-end pipeline, with each stage corresponding directly to the methodology defined in Chapter 4. Data collection, filtering, preprocessing, classification, and analysis were implemented as modular steps, ensuring consistency and reproducibility across both Reddit and YouTube corpora. Design decisions, particularly in keyword filtering and language detection, were informed by observed data quality issues and aimed to preserve relevant content while minimising noise in the final dataset.

Chapter 6: Evaluation

6.1 Evaluation Approach

I evaluated the sentiment classifier using 10-fold stratified cross-validation on the combined labelled dataset of 597 instances. Stratification ensured that each fold preserved the same class distribution as the overall dataset, which was necessary given the imbalance between positive, neutral, and negative classes.

Four evaluation metrics were used: accuracy, precision, recall, and F1-score. Accuracy provides an overall measure of correctness, but it can be misleading in imbalanced datasets. For this reason, precision, recall, and F1-score were also considered, as they provide a more detailed view of performance for each class. F1-score was particularly important, as it balances precision and recall and provides a more reliable measure when one class is underrepresented.

Both Naïve Bayes and Support Vector Machine (SVM) classifiers were evaluated in line with objectives C4 and C5, with the aim of selecting the most suitable model for full corpus prediction.

6.2 Classifier Performance

Error! Reference source not found.6.2 presents the performance of both classifiers across all evaluation metrics.

TABLE 6.2 - CLASSIFIER PERFORMANCE (10-FOLD CROSS-VALIDATION RESULTS)

Metric	Naïve Bayes	SVM (SMO)
Overall Accuracy	56.28%	**65.49%**
Kappa Statistic	0.209	**0.310**
Negative Precision	0.059	**0.167**
Negative Recall	0.043	0.043
Negative F1	0.050	**0.069**
Neutral Precision	**0.719**	0.675
Neutral Recall	0.452	**0.778**
Neutral F1	0.555	**0.723**
Positive Precision	0.497	**0.631**
Positive Recall	**0.767**	0.542
Positive F1	**0.603**	0.583
Weighted Avg F1	0.555	**0.642**

SVM achieved higher overall accuracy, weighted F1-score, and Kappa statistic, indicating stronger and more consistent performance across classes. The improvement in weighted F1-score is particularly

important, as it reflects better balance across all classes rather than performance driven by the dominant neutral class.

Naïve Bayes showed higher recall for the positive class, meaning it identified more positive instances. However, this came at the cost of lower precision, indicating a higher rate of false positives. SVM, in contrast, produced fewer positive predictions but with greater accuracy.

Based on these results, SVM was selected as the final model for full corpus prediction, as it provided a better overall balance between precision and recall and improved performance across the dataset.

6.3 Confusion Matrix Analysis

TABLE 6.3.1 - NAIVE BAYES CONFUSION MATRIX

	Predicted Negative	Predicted Neutral	Predicted Positive
Actual Negative	1	8	14
Actual Neutral	11	151	172
Actual Positive	5	51	184

TABLE 6.3.2 - SVM (SMO) CONFUSION MATRIX

	Predicted Negative	Predicted Neutral	Predicted Positive
Actual Negative	1	17	5
Actual Neutral	3	260	71
Actual Positive	2	108	130

Both classifiers performed poorly on the negative class. Out of 23 negative instances, SVM correctly identified only 1. Most negative comments were misclassified as neutral or positive. This reflects the limited representation of negative examples in the training data, which restricted the model's ability to learn distinguishing patterns.

SVM performed significantly better on the neutral class, correctly classifying 260 out of 334 instances. This is reflected in its high recall for neutral sentiment. Given that neutral is the majority class, this contributed strongly to overall accuracy.

For the positive class, Naïve Bayes achieved higher recall but lower precision, indicating that it tended to over-predict positive sentiment. SVM showed lower recall but higher precision, suggesting that it was more selective and produced more reliable positive predictions.

These results indicate that while SVM improves overall classification quality, performance remains uneven across classes, with negative sentiment being consistently under-detected.

6.4 Baseline Comparison

A majority-class baseline classifier that always predicts neutral would achieve approximately 56% accuracy, reflecting the proportion of neutral instances in the dataset. Naïve Bayes achieved 56.28%, which is only marginally above this baseline and suggests limited predictive value beyond class distribution.

SVM achieved 65.49%, representing a clear improvement over the baseline. This indicates that the model learned meaningful patterns beyond simply predicting the majority class.

The Kappa statistic provides further insight by measuring agreement beyond chance. SVM achieved a Kappa of 0.310, indicating fair agreement, while Naïve Bayes achieved 0.209, indicating only slight-to-fair agreement. Neither model reached the level typically considered strong performance, which reflects the difficulty of the task and the limitations of the dataset.

6.5 Domain Transfer Validation

The classifier was trained on a combined dataset from YouTube and Reddit. Applying a model across platforms introduces domain transfer challenges, as language use, tone, and formatting differ between platforms.

To validate that the model generalised across both sources, the classification pipeline was replicated in scikit-learn using LinearSVC and TF-IDF vectorisation. The replicated model achieved a cross-validated accuracy of 68.52% (±6.83%), which is consistent with the WEKA results.

This consistency provides confidence that the implementation is correct and that the model's performance is stable across different tools. It also suggests that combining YouTube and Reddit data did not significantly reduce classification performance, although differences in platform language remain a potential limitation.

Chapter 7: Results

7.1 Overview

This chapter presents the results of the sentiment classification and correlation analysis. The findings are reported in a structured and descriptive manner. Interpretation and evaluation of these results are presented in Chapter 8.

7.2 Sentiment Classification Results

7.2.1 YouTube Sentiment Distribution

Table presents the distribution of predicted sentiment classes for YouTube comments across the six comebacks.

TABLE 7.2.1 - YOUTUBE SENTIMENT DISTRIBUTION PER COMEBACK

Comeback	Negative	Neutral	Positive	Total	Positive Ratio	Sentiment Score
aespa – Whiplash	6	100	66	172	0.384	0.349
IVE – Rebel Heart	1	60	48	109	0.440	0.431
TWICE – Strategy	6	95	62	163	0.380	0.344
NCT DREAM – When I'm With You	1	102	76	179	0.425	0.419
ATEEZ – Ice On My Teeth	1	149	122	272	0.449	0.445
Stray Kids – Chk Chk Boom	4	67	69	140	0.493	0.464

Sentiment score is calculated as (positive − negative) / total. Stray Kids recorded the highest sentiment score (0.464), followed by ATEEZ (0.445) and IVE (0.431). aespa and TWICE recorded the lowest sentiment scores (0.349 and 0.344 respectively).

7.2.2 Reddit Sentiment Distribution

Table presents the distribution of predicted sentiment classes for Reddit comments across the six comebacks.

TABLE 7.2.2 - REDDIT SENTIMENT DISTRIBUTION PER COMEBACK

Comeback	Negative	Neutral	Positive	Total	Positive Ratio	Sentiment Score
aespa – Whiplash	110	1,742	974	2,826	0.345	0.306
IVE – Rebel Heart	23	1,042	698	1,763	0.396	0.383
TWICE – Strategy	23	1,229	962	2,214	0.434	0.424

NCT DREAM – When I'm With You	12	338	78	428	0.182	0.154
ATEEZ – Ice On My Teeth	34	2,692	1,318	4,044	0.326	0.318
Stray Kids – Chk Chk Boom	18	3,228	1,396	4,642	0.301	0.297

TWICE recorded the highest Reddit sentiment score (0.424), followed by IVE (0.383). NCT DREAM recorded the lowest sentiment score (0.154).

7.3 YouTube Engagement Metrics

Table presents the YouTube engagement metrics collected for each comeback music video.

TABLE 7.3 - YOUTUBE ENGAGEMENT METRICS PER COMEBACK

Comeback	View Count	Like Count	Comment Count	Like-to-View Ratio
aespa – Whiplash	272,860,242	2,496,666	94,955	0.00915
IVE – Rebel Heart	64,041,208	784,179	42,838	0.01225
TWICE – Strategy	146,374,901	1,970,932	204,572	0.01347
NCT DREAM – When I'm With You	15,889,568	515,104	42,525	0.03242
ATEEZ – Ice On My Teeth	98,921,178	1,062,250	68,104	0.01074
Stray Kids – Chk Chk Boom	198,946,095	3,631,709	391,720	0.01826

aespa recorded the highest view count and like count, while Stray Kids recorded the highest comment count. NCT DREAM recorded the highest like-to-view ratio despite having the lowest view count.

7.4 Correlation Results

Table presents the correlation between sentiment scores and YouTube engagement metrics across the six comebacks.

TABLE 7.4 - CORRELATION BETWEEN SENTIMENT SCORES AND YOUTUBE ENGAGEMENT METRICS (N = 6)

Sentiment Source	Performance Metric	Pearson r	p-value	Spearman ρ	p-value
YouTube sentiment	View count	-0.404	0.427	-0.086	0.872
YouTube sentiment	Like count	-0.046	0.931	0.143	0.787
YouTube sentiment	Comment count	0.193	0.714	0.086	0.872
YouTube sentiment	Like-to-view ratio	0.273	0.601	0.143	0.787
Reddit sentiment	View count	0.301	0.562	0.086	0.872

Reddit sentiment	Like count	0.208	0.693	0.029	0.957
Reddit sentiment	Comment count	0.186	0.724	0.200	0.704
Reddit sentiment	Like-to-view ratio	**-0.795**	**0.059**	-0.429	0.397

The strongest observed relationship was between Reddit sentiment score and like-to-view ratio (Pearson r = -0.795, p = 0.059). All other correlations were weak and not statistically significant.

7.5 Tableau Dashboard

The results are presented in an interactive Tableau Public dashboard comprising five visualisations: YouTube engagement metrics, YouTube sentiment distribution, Reddit sentiment distribution, YouTube sentiment versus view count, and Reddit sentiment versus like-to-view ratio.

The dashboard is available at:
https://public.tableau.com/views/KPop_Sentiment_Analysis_Dashboard/K-popSentimentDashboard

Screenshot 1 shows the dashboard interface. Additional screenshots are provided in Appendix C: Tableau Dashboard Screenshot.

Chapter 8: Discussion

8.1 Overview

This chapter analyses the key findings presented in Chapter 7, considers their implications in relation to the research question, and outlines the limitations of the study.

8.2 Classifier Performance

The SVM classifier achieved an accuracy of 65.49% on a three-class sentiment classification task, representing a clear improvement over the majority-class baseline of 56%. This indicates that the model was able to learn meaningful patterns from the data rather than relying on class distribution alone.

Performance on the negative class was notably poor, with an F1-score of 0.069. This reflects the severe class imbalance in the training data, where negative instances accounted for only 3.9% of the labelled dataset. With only 23 negative examples available, the classifier had limited opportunity to learn features associated with negative sentiment.

This imbalance is not solely a methodological issue but reflects the nature of the data itself. During comeback periods, fan discourse on YouTube and Reddit is largely positive or neutral. Negative opinions are less visible, either because they are less frequently expressed in fan spaces or because they receive lower engagement and are therefore less likely to appear in collected data. This suggests that sentiment derived from these platforms may not represent a fully balanced view of audience reception.

The comparative performance of Naïve Bayes and SVM is consistent with prior research. While some studies report stronger performance for Naïve Bayes in multilingual contexts, others find that SVM performs better on English-language social media data. The results of this project align with the latter, with SVM providing higher accuracy and a better balance between precision and recall.

8.3 Correlation Findings

The central research question examined whether social media sentiment correlates with YouTube performance metrics across K-pop comebacks. The results suggest that this relationship is limited and not statistically significant within the scope of this study.

No significant correlation was found between YouTube comment sentiment and any engagement metric. One explanation is that YouTube comments are generated after content consumption, meaning engagement metrics such as views and likes are not driven by sentiment expressed in comments. In addition, videos with higher view counts tend to attract a broader audience, introducing more varied sentiment.

For example, aespa's "Whiplash" recorded the highest view count but a relatively low sentiment score. This pattern is consistent with a wider audience base, where responses are more diverse than those within core fan communities.

The strongest observed relationship was a negative correlation between Reddit sentiment and like-to-view ratio. Although this result did not reach statistical significance, it suggests a possible inverse relationship between sentiment and engagement efficiency.

NCT DREAM provides a clear example. Despite recording the lowest Reddit sentiment score, it achieved the highest like-to-view ratio. This may reflect differences in fanbase structure, where a smaller but more engaged audience generates higher interaction relative to view count.

These findings suggest that sentiment alone does not directly explain performance metrics. Instead, engagement appears to be influenced by additional factors such as fanbase size, organisation, and behaviour. This is consistent with existing research showing that social network structure and community dynamics play a significant role in shaping popularity outcomes.

It is important to note that the observed correlations are based on a small sample size (n=6). The borderline p-value (0.059) indicates that the result is suggestive but not statistically reliable. A larger dataset would be required to determine whether this relationship holds more broadly.

8.4 Comparison with Existing Research

Previous research has shown that social media activity can be used to predict aspects of entertainment performance, particularly when considering the volume of discussion. Studies using Twitter data, for example, have found moderate correlations between mention frequency and chart performance.

This project extends that approach by focusing on sentiment rather than volume and by examining YouTube engagement metrics instead of chart rankings. The weaker correlations observed suggest that sentiment may be a less reliable predictor of performance than activity level.

One possible explanation is the nature of K-pop fandoms. Fan engagement is often coordinated through organised streaming and promotional efforts, which can increase views and interactions independently of sentiment. This reduces the extent to which engagement metrics reflect organic audience response.

As a result, the relationship between sentiment and performance in this context appears indirect and mediated by fan behaviour rather than being directly causal.

8.5 Limitations

Several limitations should be considered when interpreting the results of this study.

Sample size
The analysis is based on six comebacks, resulting in very low statistical power. Correlation results should be treated as exploratory rather than conclusive.

Cumulative YouTube metrics
Engagement metrics were collected as cumulative totals rather than within a fixed time window. This introduces temporal bias, as earlier releases have had more time to accumulate views and interactions.

Class imbalance
The low proportion of negative examples limited the classifier's ability to detect negative sentiment, reducing performance for that class.

YouTube sampling method
YouTube comments were sampled sequentially rather than randomly, which may introduce ordering bias.

Single annotator
All sentiment labels were assigned by a single annotator. No inter-annotator agreement was calculated, which may affect reliability.

Reddit corpus scope
Reddit data includes general discussion within the comeback window and is not restricted exclusively to comments about the specific release.

English-only analysis
Non-English comments were excluded, meaning that a significant portion of global fan discourse is not represented in the dataset.

Domain mismatch
The classifier was trained on combined YouTube and Reddit data, despite differences in language style between platforms. This may affect classification performance when applied to each dataset independently.

Chapter 9: Conclusion

9.1 Extent to Which Objectives Were Met

This chapter reflects on the extent to which the project objectives were achieved and outlines directions for future work.

C1. **Reddit data collection**
Fully met. Reddit comments were collected from nine subreddits for all six comebacks using the Pushshift archive dumps within 14-day windows from each release date. Additional keyword and language filtering steps were introduced during development to improve data quality.

C2. **YouTube engagement metrics**
Fully met. View count, like count, comment count, and like-to-view ratio were collected for all six comeback music videos using the YouTube Data API v3.

C3. **Preprocessing**
Fully met. A consistent preprocessing pipeline was applied across both Reddit and YouTube datasets, including encoding normalisation, tokenisation, stopword removal, and stemming.

C4. **Manual labelling**
Substantially met. A total of 597 comments were labelled across both platforms using a predefined codebook. The target of 50 YouTube comments per comeback was achieved, while Reddit sampling averaged slightly below the target due to deduplication.

C5. **Classifier training and evaluation**
Fully met. Naïve Bayes and Support Vector Machine classifiers were trained and evaluated using 10-fold cross-validation. Performance metrics were reported, and SVM was selected as the best-performing model.

C6. **Full corpus classification**
Partially met. The selected classifier was applied to the full datasets, and sentiment distributions were generated. A technical limitation in WEKA required the classification pipeline to be replicated in scikit-learn. The replicated results were consistent, supporting the validity of the implementation.

C7. **Correlation analysis**
Fully met. Pearson and Spearman correlation coefficients were calculated between sentiment scores and YouTube engagement metrics, with p-values reported.

C8. **Tableau dashboard**
Fully met. An interactive dashboard was developed and published on Tableau Public: https://public.tableau.com/views/KPop_Sentiment_Analysis_Dashboard/K-popSentimentDashboard

Advanced objectives (A1, A2, A3)

Not pursued. A1 was not feasible due to lack of access to the Twitter API. A2 and A3 were deferred due to time constraints following adjustments to the core methodology. These remain valid extensions for future work.

9.2 Response to the Research Question

The research question asked whether social media sentiment toward K-pop comebacks correlates with YouTube performance metrics. The findings indicate that this relationship is weak and inconsistent within the scope of this study.

No statistically significant correlation was found between YouTube comment sentiment and any engagement metric. A negative correlation was observed between Reddit sentiment and like-to-view ratio, although this result did not reach conventional significance. This suggests that higher sentiment does not necessarily translate into higher engagement efficiency.

Overall, the results do not support a direct relationship between positive fan sentiment and YouTube performance. Instead, engagement appears to be influenced by additional factors such as fanbase size, audience composition, and coordinated fan activity.

9.3 Reflection

Several aspects of the methodology could be improved in future work.

The use of sequential sampling for YouTube comments may have introduced ordering bias. A random stratified sampling approach would provide a more representative dataset. The class imbalance in the labelled data, particularly the limited number of negative examples, reduced the classifier's ability to detect negative sentiment. Increasing the size of the labelled dataset or applying resampling techniques could improve model performance.

The use of cumulative YouTube metrics is a key limitation. Because metrics were not collected within a fixed time window, comparisons between comebacks are affected by differences in release timing. Collecting metrics at a consistent interval after release would allow for more accurate comparison.

Despite these limitations, the project demonstrates that an end-to-end sentiment analysis pipeline can be implemented using publicly available data and standard tools. The combination of Pushshift data and the YouTube Data API provided sufficient coverage to support the analysis.

9.4 Future Work

Several extensions would strengthen this study.

Increasing the number of comebacks analysed would improve statistical power and allow more reliable correlation analysis. Including multilingual data would provide a more representative view of global fan sentiment. Temporal analysis within the release window could identify whether early sentiment is more predictive than later discussion. Further work could also examine the impact of coordinated streaming behaviour, which may influence engagement metrics independently of sentiment.

Bibliography

Aprinastya, R. *et al.* (2024) 'Comparative Analysis of Naïve Bayes Classifier and Support Vector Machine for Multilingual Sentiment Analysis: Insights from Genshin Impact User Reviews', *JUSIFO (Jurnal Sistem Informasi)*, 10(2), pp. 117–126.

Asur, S. and Huberman, B.A. (2010) 'Predicting the future with social media', *Proceedings of the 2010 IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology*. Toronto: IEEE, pp. 492–499.

BCS (2022) *BCS Code of Conduct*. Professional Standard Version 8. London, UK: BCS, The Chartered Institute for IT. Available at: https://www.bcs.org/media/2211/bcs-code-of-conduct.pdf.

Jung, S. (2011) 'K-pop, Indonesian fandom, and social media | Jung | Transformative Works and Cultures.', *Transformative Works & Cultures*, 8.

Liu, B. (2012) 'Sentiment Analysis and Opinion Mining'.

Millennianita, F., Athiyah, U. and Muhammad, A.W. (2024) 'Comparison of naive bayes classifier and support vector machine methods for sentiment classification of responses to bullying cases on Twitter', *Journal of Mechatronics and Artificial Intelligence*, 1(1), pp. 11–26.

Pang, B. and Lee, L. (2008) *Opinion mining and sentiment analysis*.

Reisz, N., Servedio, V.D.P. and Thurner, S. (2024) 'Quantifying the impact of homophily and influencer networks on song popularity prediction', *Scientific Reports*, 14(1), p. 8929. Available at: https://doi.org/10.1038/s41598-024-58969-w.

Rui, H., Liu, Y. and Whinston, A.B. (2013) 'Whose and what chatter matters? The effect of tweets on movie sales', *Decision Support Systems*, 55(4), pp. 863–870.

Tsiara, E. and Tjortjis, C. (2020) 'Using Twitter to Predict Chart Position for Songs', in I. Maglogiannis, L. Iliadis, and E. Pimenidis (eds) *Artificial Intelligence Applications and Innovations*. Springer International Publishing, pp. 62–72.

Zhang, J. *et al.* (2025) 'A Study on the Impact of Social Media on K-pop Fans' Behavior and Emotions: A Case Study of Chinese Fans', *Interdisciplinary Humanities and Communication Studies*, 1(3).

Appendices

Each appendix is referenced in the main body of the report and includes supporting material that is not required within the core chapters but is necessary for transparency, reproducibility, and verification of the work.

Appendix A: Labelling Codebook

APPENDIX A: LABELLING CODEBOOK

Introduction

This appendix contains the formal labelling codebook used to guide manual sentiment annotation of YouTube and Reddit comments. Labels were assigned as positive, negative, or neutral in accordance with the decision rules defined in the codebook.

Content

Labelling Codebook

```
# Labelling Codebook

**Project**: K-pop Comeback Sentiment Analysis
**Author**: Miracle Okoi-Obuli
**Date**: April 2026
**Scope**: Manual sentiment labelling of YouTube and Reddit comments for
six K-pop comebacks.

\---

## 1\. Purpose

This codebook defines the rules used to assign sentiment labels to
comments sampled from YouTube and Reddit during the manual labelling phase
of the project. Its purpose is to make the labelling process transparent,
consistent, and reproducible. The labelled dataset is used to train the
sentiment classifier in WEKA (Naïve Bayes and Support Vector Machine),
which is subsequently applied to the full preprocessed corpus.

\---

## 2\. Label Classes

Each comment is assigned exactly one of three labels.

### 2.1 Positive

A comment is labelled **positive** if it expresses approval, enthusiasm,
affection, admiration, excitement, or praise directed at the group, its
members, the song, the music video, the performance, the concept, the
choreography, the visuals, or any aspect of the comeback.

Indicators include:

* Explicit expressions of love, enjoyment, or excitement ("this is
amazing", "obsessed with this song", "they ate").
```

* Complimentary observations about members or performance ("Wonyoung's vocals are incredible").
* Supportive rallying messages ("let's stream this for them", "they deserve a win").
* Enthusiastic emoji use where the accompanying text is positive or neutral (🔥 💖 😍 ❤️ 🥹 👑).
* Fan-coded positivity such as "mother", "slayed", "ate and left no crumbs", "PAK incoming", or group-specific hype phrases.

2.2 Negative

A comment is labelled **negative** if it expresses disapproval, disappointment, criticism, dislike, frustration, or hostility directed at the group, its members, the song, the music video, the performance, the concept, the label, or any aspect of the comeback.

Indicators include:

* Explicit statements of disappointment or dislike ("this is boring", "worst title track", "I expected more").
* Criticism of production, concept, choreography, vocals, or visuals.
* Unfavourable comparisons to previous releases or other groups, where the comparison is made as criticism.
* Expressions of frustration with the group's label, management, or promotion strategy when directed at this comeback.
* Sarcasm or backhanded remarks whose clear intent is critical, even when surface wording sounds positive.

2.3 Neutral

A comment is labelled **neutral** if it does not express a clear positive or negative sentiment toward the comeback. This includes factual, informational, observational, or ambiguous comments.

Indicators include:

* Factual statements or announcements ("MV drops at 6pm KST", "released on 13 January").
* Questions without evaluative content ("what time is the comeback showcase?", "is this a pre-release?").
* Observations about unrelated topics, streaming strategy, or industry context that do not themselves evaluate the comeback.
* Short comments with no clear evaluative content ("first", "here", "hi").
* Comments whose sentiment cannot be determined with reasonable confidence.

\---

3\. Decision Rules for Edge Cases

The following rules apply when a comment is not clearly one label.

Mixed sentiment. If a comment contains both positive and negative signals, label based on the dominant sentiment. Count the number and

strength of positive vs. negative signals. If the comment is genuinely balanced and neither side clearly dominates, label as neutral.

Sarcasm. Label based on perceived intent, not surface wording. A comment such as "wow, so original" used sarcastically to mock a repetitive concept is negative, not positive. Rely on context, punctuation (exaggerated capitalisation, excessive punctuation), and adjacent phrases to identify sarcasm.

Praise for one member, criticism of another. If a comment praises one member while criticising another in the same group, and the comment is about this comeback, label based on the dominant sentiment toward the comeback as a whole. If the comment is neutral toward the comeback itself but evaluative only about individual members, lean neutral.

Comments about streaming or charts. Rally messages ("stream for them!", "let's get them to number one") are positive. Neutral observations about chart performance ("they're at number 5 on Melon right now") are neutral. Criticism of chart performance or fan behaviour ("flop era", "nobody is streaming") is negative.

Emoji-only comments. Label based on the clear valence of the emoji set. 🔥🔥🔥 or 💖 alone is positive. 💀 used ironically is typically negative. 😐 or ❓ alone is neutral. If uncertain, neutral.

Non-comeback content. If a sampled comment discusses something unrelated to the comeback (e.g., a tangential reply about another group's scandal), label based on the sentiment of that content if clearly evaluative, or neutral if not. Do not discard; the sample is fixed.

Short or ambiguous comments. If a comment is too short or too ambiguous to confidently determine sentiment, label as neutral. Do not default to positive simply because K-pop fan discourse is often positive.

Multilingual residue. If a comment passed the English filter but contains a significant non-English portion that changes meaning, label only on the basis of the English portion. If the English portion alone does not carry clear sentiment, label as neutral.

\---

4\. Process

1. Open the sample file (`reddit\_labelling\_template.csv` or `youtube\_labelling\_template.csv`) in Excel or an equivalent spreadsheet tool.
2. Label each row by entering `positive`, `negative`, or `neutral` in the `label` column (exact lowercase values, consistent across all 600 rows).
3. Save the labelled file at regular intervals to avoid data loss.
4. Work in focused batches of 50 comments at a time. Labelling judgement deteriorates after roughly 30 minutes of continuous work; short breaks preserve consistency.
5. When a decision is genuinely difficult, pause, re-read the codebook, and make a best judgement. Do not skip rows.

\---

5\. Quality and Limitations

This labelling was conducted by a single annotator (the project author), which is a known limitation of small-scale undergraduate projects. Inter-annotator agreement statistics such as Cohen's kappa cannot therefore be reported. To reduce within-annotator drift, all labels were assigned within a single week following this codebook, and the codebook was not revised after labelling commenced.

The three-class scheme (positive, negative, neutral) was chosen to align with WEKA's supported classification tasks and with the Interim Progress Report commitment. Finer-grained schemes (e.g. five-point Likert or emotion-specific categories) were rejected as impractical for the project scale and timeline.

Sentiment labelling of fan-generated content is inherently subjective and fandom-literate judgement is required to interpret slang, sarcasm, and in-group references correctly. The author is familiar with K-pop fan culture, which supports accurate interpretation of such content but also introduces a known fan-perspective bias that is acknowledged in the Discussion chapter of the Final Report.

Appendix B: Source Code

Introduction

Full source code is also available in the project GitHub repository at https://github.com/mimiokoiobuli/kpop-sentiment-analysis.

The code is also submitted separately as the software artefact on Canvas.

Content

Script 1: reddit_collection.py

```
"""
reddit_collection.py
--------------------
Collects Reddit comments from Pushshift monthly archive dumps for six K-
pop
comebacks during their release windows.

Background
    The original methodology (documented in the Interim Progress Report,
    March 2026) planned to use the Reddit API via PRAW. After the Reddit
    API access request was not approved in time, the project pivoted to
    the publicly available Pushshift monthly archive dumps, distributed
    as .zst-compressed newline-delimited JSON via Academic Torrents.

    This script replaces the earlier collection script (lost from local
    storage before it was committed to git). It reproduces the same
    methodology that was used to generate the raw CSVs already saved
    under data/raw/reddit/ and is included in the code appendix as the
    canonical reference for the Reddit collection step.

Methodology
    For each of the six target comebacks, Reddit comments are pulled
    from nine K-pop-related subreddits within a 14-day window starting
    on the release date (day 0 through day +13 inclusive).

    Filtering rules per comment:
        1. Subreddit must be in the target list.
        2. created_utc must fall within the comeback's 14-day window.
        3. body must not be empty, '[deleted]', or '[removed]'.
        4. If the comment is posted in the comeback group's *own*
           subreddit (e.g., r/Aespa for the Whiplash comeback), it is
           kept without a keyword filter. Activity in a group's home
           subreddit during its release window is assumed to be
           dominated by that comeback.
        5. Otherwise, the comment body must contain either the group
           name or the song title (case-insensitive substring match).

    Duplicate comment IDs across dump files are removed.

Output
    One CSV per comeback, written to data/raw/reddit/, with columns:
```

```
        id, subreddit, text, score, created_utc, comeback

Input
    Pushshift monthly comment dumps named 'RC_YYYY-MM.zst' placed in
    the directory referenced by DUMP_DIR below.

Usage
    python reddit_collection.py

Dependencies
    pip install zstandard
"""

from __future__ import annotations

import csv
import io
import json
from datetime import datetime, timedelta
from pathlib import Path

import zstandard as zstd


# ---------------------------------------------------------------------
---
# Configuration
# ---------------------------------------------------------------------
---

# Comebacks under study. Release dates confirmed against each group's
# official release; keyword terms are lowercase substrings matched against
# the comment body.

COMEBACKS = [
    {
        "label":        "StrayKids_ChkChkBoom",
        "group":        "stray kids",
        "song":         "chk chk boom",
        "release_date": datetime(2024, 7, 19),
        "own_sub":      "straykids",
    },
    {
        "label":        "aespa_Whiplash",
        "group":        "aespa",
        "song":         "whiplash",
        "release_date": datetime(2024, 10, 21),
        "own_sub":      "Aespa",
    },
    {
        "label":        "NCTDREAM_WhenImWithYou",
        "group":        "nct dream",
        "song":         "when i'm with you",
        "release_date": datetime(2024, 11, 11),
```

```
        "own_sub":      "NCTDream",
    },
    {
        "label":        "ATEEZ_IceOnMyTeeth",
        "group":        "ateez",
        "song":         "ice on my teeth",
        "release_date": datetime(2024, 11, 15),
        "own_sub":      "ATEEZ",
    },
    {
        "label":        "TWICE_Strategy",
        "group":        "twice",
        "song":         "strategy",
        "release_date": datetime(2024, 12, 6),
        "own_sub":      "twice",
    },
    {
        "label":        "IVE_RebelHeart",
        "group":        "ive",
        "song":         "rebel heart",
        "release_date": datetime(2025, 1, 13),
        "own_sub":      "IVE",
    },
]

# 14-day window: [release_date, release_date + WINDOW_DAYS)
WINDOW_DAYS = 14

# Subreddits searched for every comeback. Comparisons are case-
insensitive.
SUBREDDITS = {
    "kpop", "kpopthoughts", "unpopularkpopopinions",
    "straykids", "aespa", "NCTDream", "ATEEZ", "twice", "IVE",
}
SUBREDDITS_LC = {s.lower() for s in SUBREDDITS}

# Paths. Absolute Windows paths so the script runs from any working
directory.
# The r"" prefix makes these raw strings so backslashes are treated
literally.
DUMP_DIR   = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit\comments")
OUTPUT_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# CSV schema (order matters: matches existing raw files).
OUTPUT_FIELDS = ["id", "subreddit", "text", "score", "created_utc",
"comeback"]


# ------------------------------------------------------------------------
---
# Pushshift .zst streaming
```

```
# ----------------------------------------------------------------------
---

def read_zst_ndjson(path: Path):
    """
    Yield one JSON object per line from a zstd-compressed ndjson dump.
    The large max_window_size is required because Pushshift dumps are
    compressed with long-range mode.
    """
    dctx = zstd.ZstdDecompressor(max_window_size=2 ** 31)
    with open(path, "rb") as f:
        with dctx.stream_reader(f) as reader:
            text_stream = io.TextIOWrapper(reader, encoding="utf-8")
            for line in text_stream:
                line = line.strip()
                if not line:
                    continue
                try:
                    yield json.loads(line)
                except json.JSONDecodeError:
                    # Skip the occasional malformed line rather than
                    # aborting the whole dump.
                    continue


# ----------------------------------------------------------------------
---
# Filtering
# ----------------------------------------------------------------------
---

def keep_comment(obj: dict, comeback: dict) -> bool:
    """Apply subreddit, date-window, body, and keyword filters."""
    sub = obj.get("subreddit", "") or ""
    if sub.lower() not in SUBREDDITS_LC:
        return False

    # Date window
    try:
        ts = int(obj.get("created_utc", 0))
    except (ValueError, TypeError):
        return False
    comment_date = datetime.utcfromtimestamp(ts)
    start = comeback["release_date"]
    end   = start + timedelta(days=WINDOW_DAYS)
    if not (start <= comment_date < end):
        return False

    # Body validity
    body = obj.get("body") or ""
    body_l = body.strip().lower()
    if not body_l or body_l in ("[deleted]", "[removed]"):
        return False
```

```
    # In the group's own subreddit, keep every valid comment in-window.
    if sub.lower() == comeback["own_sub"].lower():
        return True

    # Elsewhere, require a keyword match.
    return comeback["group"] in body_l or comeback["song"] in body_l


# ------------------------------------------------------------------------
---
# Collection loop
# ------------------------------------------------------------------------
---

def months_covering(start: datetime, end: datetime):
    """Yield (year, month) pairs covering the date range inclusively."""
    y, m = start.year, start.month
    while (y, m) <= (end.year, end.month):
        yield y, m
        if m == 12:
            y, m = y + 1, 1
        else:
            m += 1


def collect_for_comeback(comeback: dict) -> list[dict]:
    """Return the filtered, de-duplicated comments for one comeback."""
    start = comeback["release_date"]
    end   = start + timedelta(days=WINDOW_DAYS)
    rows  = []
    seen  = set()

    for year, month in months_covering(start, end):
        dump = DUMP_DIR / f"RC_{year}-{month:02d}.zst"
        if not dump.exists():
            print(f"  [warn] Missing dump for {year}-{month:02d}: {dump}")
            continue

        print(f"  Reading {dump.name} ...")
        for obj in read_zst_ndjson(dump):
            if not keep_comment(obj, comeback):
                continue
            cid = obj.get("id")
            if not cid or cid in seen:
                continue
            seen.add(cid)
            rows.append({
                "id":          cid,
                "subreddit":   obj.get("subreddit", ""),
                "text":        obj.get("body", ""),
                "score":       obj.get("score", 0),
                "created_utc": obj.get("created_utc", 0),
                "comeback":    comeback["label"],
            })
```

```
    return rows


def main() -> None:
    for cb in COMEBACKS:
        print(f"\n>>> Collecting {cb['label']} ...")
        rows = collect_for_comeback(cb)
        out_path = OUTPUT_DIR / f"{cb['label']}_reddit_raw.csv"
        with open(out_path, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=OUTPUT_FIELDS)
            writer.writeheader()
            writer.writerows(rows)
        print(f"  Saved {len(rows)} comments -> {out_path}")


if __name__ == "__main__":
    main()
```

Script 2: reddit_filter.py

```
 """
reddit_filter.py
----------------
Post-hoc keyword filter applied to the six raw Reddit CSVs produced by
reddit_collection.py. Replaces the initial case-insensitive substring
match with a stricter, word-boundary-aware ruleset that eliminates
false positives caused by group names colliding with common English
words (notably IVE vs. the contraction "I've", and TWICE vs. the
adverb "twice").

Rationale
    The original collection script used `group_name in text.lower()`.
    This worked well for distinctive tokens (aespa, ATEEZ, stray kids,
    nct dream) but introduced substantial noise for IVE and TWICE.
    Audit of the raw IVE data showed that comments pulled from non-IVE
    subreddits matched "ive" as a substring of common words (I've,
    received, alive, give) in the large majority of cases.

    This filter is applied symmetrically to all six comebacks so the
    refinement does not bias one group relative to the others.

Filter rules (applied only to comments outside the comeback's own sub)
    A comment is kept if any one of:
        1. The song title phrase matches (case-insensitive). If the
           title is itself a common English word or phrase (Strategy,
           When I'm With You), the comment must also contain the
           group name token or a recognised member name.
        2. The group name pattern matches. For all-caps stylized
           names (IVE, TWICE), matching is case-sensitive to
           distinguish "IVE" from "I've" / "ive".
        3. A member name (word-boundary, case-insensitive) appears.
```

```
Own-sub comments are always kept. Topical relevance is implicit when
a user posts in the group's own subreddit.

Input
    Six raw CSVs in 01_raw_data/reddit/

Output
    Six cleaned CSVs in 01_raw_data/reddit/cleaned/
    Plus a _filter_summary.csv table of rows kept vs. dropped.
"""

from __future__ import annotations

import re
import pandas as pd
from pathlib import Path


# ---------------------------------------------------------------------------
---
# Configuration
# ---------------------------------------------------------------------------
---

INPUT_DIR  = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit")
OUTPUT_DIR = INPUT_DIR / "cleaned"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# Per-comeback filter specification.
#   own_sub:        subreddit name (case-insensitive comparison)
#   song_pattern:   regex for the song title
#   song_ambiguous: True if the song title is a common English word/phrase
#                   and therefore requires additional context to match
#   group_pattern:  regex for the group name
#   group_case:     True = case-sensitive match (for all-caps
stylizations)
#   members:        list of member names (word-boundary, case-insensitive)
FILTERS = {
    "IVE_RebelHeart": {
        "file":           "IVE_RebelHeart_reddit_raw.csv",
        "own_sub":        "ive",
        "song_pattern":   r"rebel\s+heart",
        "song_ambiguous": False,
        "group_pattern":  r"\bIVE\b",
        "group_case":     True,
        "members":        ["yujin", "gaeul", "rei", "wonyoung", "liz",
"leeseo"],
    },
    "TWICE_Strategy": {
        "file":           "TWICE_Strategy_reddit_raw.csv",
        "own_sub":        "twice",
        "song_pattern":   r"\bstrategy\b",
        "song_ambiguous": True,
```

```
        "group_pattern":  r"\bTWICE\b",
        "group_case":     True,
        "members":        ["nayeon", "jeongyeon", "momo", "sana", "jihyo",
                           "mina", "dahyun", "chaeyoung", "tzuyu"],
    },
    "aespa_Whiplash": {
        "file":           "aespa_Whiplash_reddit_raw.csv",
        "own_sub":        "aespa",
        "song_pattern":   r"\bwhiplash\b",
        "song_ambiguous": False,
        "group_pattern":  r"\baespa\b",
        "group_case":     False,
        "members":        ["karina", "giselle", "winter", "ningning"],
    },
    "StrayKids_ChkChkBoom": {
        "file":           "StrayKids_ChkChkBoom_reddit_raw.csv",
        "own_sub":        "straykids",
        "song_pattern":   r"chk\s*chk\s*boom",
        "song_ambiguous": False,
        "group_pattern":  r"\bstray\s*kids\b|\bSKZ\b",
        "group_case":     False,
        "members":        ["bang chan", "bangchan", "lee know", "leeknow",
                           "changbin", "hyunjin", "felix", "seungmin",
                           "han jisung"],
    },
    "NCTDREAM_WhenImWithYou": {
        "file":           "NCTDREAM_WhenImWithYou_reddit_raw.csv",
        "own_sub":        "nctdream",
        "song_pattern":   r"when\s+i'?m\s+with\s+you",
        "song_ambiguous": True,
        "group_pattern":  r"\bnct\s*dream\b",
        "group_case":     False,
        "members":        ["renjun", "jeno", "haechan", "jaemin",
                           "chenle", "jisung"],
    },
    "ATEEZ_IceOnMyTeeth": {
        "file":           "ATEEZ_IceOnMyTeeth_reddit_raw.csv",
        "own_sub":        "ateez",
        "song_pattern":   r"ice\s+on\s+my\s+teeth",
        "song_ambiguous": False,
        "group_pattern":  r"\bateez\b",
        "group_case":     False,
        "members":        ["hongjoong", "seonghwa", "yunho", "yeosang",
                           "wooyoung", "jongho", "mingi"],
    },
}


# ---------------------------------------------------------------------------
---
# Matching logic
# ---------------------------------------------------------------------------
---
```

```
def match_group(text: str, spec: dict) -> bool:
    flags = 0 if spec["group_case"] else re.IGNORECASE
    return bool(re.search(spec["group_pattern"], text, flags))


def match_song(text: str, spec: dict) -> bool:
    return bool(re.search(spec["song_pattern"], text, re.IGNORECASE))


def match_member(text: str, spec: dict) -> bool:
    for m in spec["members"]:
        if re.search(rf"\b{re.escape(m)}\b", text, re.IGNORECASE):
            return True
    return False


def is_true_mention(text: str, spec: dict) -> bool:
    if not isinstance(text, str) or not text:
        return False

    if match_song(text, spec):
        if spec["song_ambiguous"]:
            if match_group(text, spec) or match_member(text, spec):
                return True
        else:
            return True

    if match_group(text, spec):
        return True

    if match_member(text, spec):
        return True

    return False


# ------------------------------------------------------------------------
---
# Main
# ------------------------------------------------------------------------
---

def filter_file(label: str, spec: dict) -> dict:
    path = INPUT_DIR / spec["file"]
    df = pd.read_csv(path)
    total = len(df)

    own_mask = df["subreddit"].str.lower() == spec["own_sub"].lower()
    other_df = df[~own_mask].copy()
    kept_other = other_df["text"].apply(lambda t: is_true_mention(t,
spec))

    cleaned = pd.concat([df[own_mask], other_df[kept_other]],
ignore_index=True)
```

```
    out_path = OUTPUT_DIR / spec["file"].replace("_raw.csv",
"_cleaned.csv")
    cleaned.to_csv(out_path, index=False)

    return {
        "comeback":     label,
        "total_raw":    total,
        "own_sub_kept": int(own_mask.sum()),
        "other_raw":    int((~own_mask).sum()),
        "other_kept":   int(kept_other.sum()),
        "total_clean":  len(cleaned),
        "dropped":      total - len(cleaned),
        "drop_pct":     round(100 * (total - len(cleaned)) / total, 1),
        "output":       out_path.name,
    }


def main() -> None:
    summary = [filter_file(label, spec) for label, spec in
FILTERS.items()]
    summary_df = pd.DataFrame(summary)

    print("\n=== FILTER SUMMARY ===")
    print(summary_df.to_string(index=False))

    summary_path = OUTPUT_DIR / "_filter_summary.csv"
    summary_df.to_csv(summary_path, index=False)
    print(f"\nSummary written to {summary_path}")


if __name__ == "__main__":
    main()
```

Script 3: language_filter.py

```
 """
language_filter.py
------------------
Filters the cleaned Reddit CSVs (and YouTube CSVs) down to English-only
comments for downstream preprocessing, manual labelling, and
classification.

Rationale
    The Interim Progress Report (March 2026) committed the methodology to
    English-language content only. K-pop fan comments across both
platforms
    regularly contain Korean (in Hangul and in romanised form),
Indonesian,
    Spanish, Thai, Turkish, and other languages. Including these in the
    corpus would produce garbage features for an English-trained
classifier.
```

```
    The `langdetect` library is the standard Python tool for this task but
    is known to be unreliable on short text (it reports high confidence on
    obviously-wrong guesses like "Wow." -> Polish, "Rebel Heart slaps"
    -> Dutch). Because fan comments are overwhelmingly short, a naive
    langdetect filter would discard a substantial amount of genuine
English
    content.

Filter logic
    1. Non-Latin script check: if a comment contains substantial Hangul,
       CJK, Cyrillic, Thai, Arabic, or Hebrew characters, it is flagged
       as non-English and excluded.
    2. Very short comments (under 15 words) are kept by default. The
       small amount of non-English noise this admits is preferable to
       the high rate of false-negative English exclusions langdetect
       produces on short text.
    3. For comments of 15 words or more, langdetect is consulted. Only
       comments detected as English with confidence >= 0.80 are kept.

    This favours recall (keeping genuine English) over precision
    (excluding every non-English comment), which is the right trade-off
    for a small training corpus.

Input
    Cleaned CSVs from 01_raw_data/reddit/cleaned/
    YouTube raw CSVs from their existing location (adjust YT_INPUT_DIR)

Output
    English-only CSVs in 01_raw_data/reddit/cleaned_en/
    A summary table reporting rows kept vs. dropped per file.
"""

from __future__ import annotations

import re
import pandas as pd
from pathlib import Path
from langdetect import detect_langs, DetectorFactory, LangDetectException


# ----------------------------------------------------------------------------
---
# Configuration
# ----------------------------------------------------------------------------
---

# Deterministic langdetect behaviour across runs
DetectorFactory.seed = 0

REDDIT_INPUT_DIR  = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit\cleaned")
REDDIT_OUTPUT_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit\cleaned_en")
REDDIT_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

```
# YouTube data is stored as a single combined CSV rather than per-comeback
# files. If YT_INPUT_FILE is set to None, YouTube filtering is skipped.
YT_INPUT_FILE  = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\youtube\youtube_comments_raw.csv")
YT_OUTPUT_FILE = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\youtube\youtube_comments_en.csv")
YT_TEXT_COL    = "comment"
YT_GROUP_COL   = "group_comeback"

LONG_TEXT_WORDS_THRESHOLD = 15
LANGDETECT_CONFIDENCE     = 0.80

# Unicode ranges for non-Latin scripts common in K-pop fan comments
NON_LATIN_RANGES = [
    (0xAC00, 0xD7AF),   # Hangul syllables
    (0x1100, 0x11FF),   # Hangul Jamo
    (0x3130, 0x318F),   # Hangul compat
    (0x3040, 0x309F),   # Hiragana
    (0x30A0, 0x30FF),   # Katakana
    (0x4E00, 0x9FFF),   # CJK unified ideographs
    (0x0400, 0x04FF),   # Cyrillic
    (0x0E00, 0x0E7F),   # Thai
    (0x0600, 0x06FF),   # Arabic
    (0x0590, 0x05FF),   # Hebrew
]


# ------------------------------------------------------------------------
---
# Filtering logic
# ------------------------------------------------------------------------
---

def non_latin_char_ratio(text: str) -> float:
    """Return the fraction of characters that fall in non-Latin
scripts."""
    if not text:
        return 0.0
    non_latin = 0
    total = 0
    for ch in text:
        cp = ord(ch)
        if ch.isspace() or not ch.isalpha():
            continue
        total += 1
        for lo, hi in NON_LATIN_RANGES:
            if lo <= cp <= hi:
                non_latin += 1
                break
    if total == 0:
        return 0.0
    return non_latin / total
```

```
def is_english(text: str) -> bool:
    """Decide whether a comment should be kept as English content."""
    if not isinstance(text, str) or not text.strip():
        return False

    # Rule 1: heavy non-Latin script -> drop
    if non_latin_char_ratio(text) >= 0.20:
        return False

    # Rule 2: short text -> keep (langdetect unreliable)
    word_count = len(text.split())
    if word_count < LONG_TEXT_WORDS_THRESHOLD:
        return True

    # Rule 3: longer text -> consult langdetect with confidence threshold
    try:
        langs = detect_langs(text)
        top = langs[0]
        return top.lang == "en" and top.prob >= LANGDETECT_CONFIDENCE
    except LangDetectException:
        # langdetect gave up (e.g. pure emoji/symbol content). Default to
        # keeping, since the non-Latin check already ran.
        return True


# ------------------------------------------------------------------------
---
# File processing
# ------------------------------------------------------------------------
---

def filter_csv(in_path: Path, out_path: Path, text_col: str = "text") ->
dict:
    df = pd.read_csv(in_path)
    total = len(df)
    if text_col not in df.columns:
        raise ValueError(f"{in_path.name}: no '{text_col}' column")

    mask = df[text_col].apply(is_english)
    kept = df[mask].copy()
    kept.to_csv(out_path, index=False)

    return {
        "file":    in_path.name,
        "total":   total,
        "kept":    int(mask.sum()),
        "dropped": total - int(mask.sum()),
        "drop_pct": round(100 * (total - int(mask.sum())) / total, 1)
                    if total else 0.0,
        "output":  out_path.name,
    }
```

```
def filter_youtube_single_file() -> list:
    """
    Filter the combined YouTube CSV and report per-comeback drop rates.
    Returns a list of summary dicts (one per comeback + one total row).
    """
    df = pd.read_csv(YT_INPUT_FILE)
    if YT_TEXT_COL not in df.columns:
        raise ValueError(f"{YT_INPUT_FILE.name}: no '{YT_TEXT_COL}'
column")

    print(f"Filtering YouTube: {YT_INPUT_FILE.name} ...")
    mask = df[YT_TEXT_COL].apply(is_english)
    kept = df[mask].copy()

    YT_OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)
    kept.to_csv(YT_OUTPUT_FILE, index=False)

    rows = []
    for cb, sub in df.groupby(YT_GROUP_COL):
        sub_mask = sub[YT_TEXT_COL].apply(is_english)
        total = len(sub)
        k = int(sub_mask.sum())
        rows.append({
            "file":     f"youtube:{cb}",
            "total":    total,
            "kept":     k,
            "dropped":  total - k,
            "drop_pct": round(100 * (total - k) / total, 1) if total else
0.0,
            "output":   YT_OUTPUT_FILE.name,
        })
    # Combined row
    rows.append({
        "file":     "youtube:TOTAL",
        "total":    len(df),
        "kept":     int(mask.sum()),
        "dropped":  len(df) - int(mask.sum()),
        "drop_pct": round(100 * (len(df) - int(mask.sum())) / len(df), 1)
if len(df) else 0.0,
        "output":   YT_OUTPUT_FILE.name,
    })
    return rows


def main() -> None:
    summary = []

    # Reddit cleaned CSVs
    for in_path in sorted(REDDIT_INPUT_DIR.glob("*_cleaned.csv")):
        out_path = REDDIT_OUTPUT_DIR /
in_path.name.replace("_cleaned.csv", "_en.csv")
        print(f"Filtering Reddit: {in_path.name} ...")
        summary.append(filter_csv(in_path, out_path, text_col="text"))
```

```
    # YouTube combined CSV
    if YT_INPUT_FILE is not None and YT_INPUT_FILE.exists():
        summary.extend(filter_youtube_single_file())
    else:
        print(f"YouTube file not found at {YT_INPUT_FILE}; skipping
YouTube.")

    summary_df = pd.DataFrame(summary)
    print("\n=== LANGUAGE FILTER SUMMARY ===")
    print(summary_df.to_string(index=False))

    summary_path = REDDIT_OUTPUT_DIR / "_language_filter_summary.csv"
    summary_df.to_csv(summary_path, index=False)
    print(f"\nSummary written to {summary_path}")


if __name__ == "__main__":
    main()
```

Script 4: preprocess.py

```
 """
preprocess.py
-------------
Preprocesses the labelled YouTube and Reddit corpora and the full
English-filtered corpora for WEKA sentiment classification.

Steps applied to every comment:
    1. Encoding normalisation (fixes â€™ and similar UTF-8 mojibake
       common in Pushshift Reddit dumps)
    2. Lowercase
    3. URL removal
    4. Emoji and non-ASCII removal
    5. Number removal (digits stripped)
    6. Punctuation removal
    7. Tokenisation (whitespace split after cleaning)
    8. Stopword removal (NLTK English stopwords)
    9. Stemming (Porter stemmer, matching standard NLP pipeline)
    10. Rejoin to cleaned string

This pipeline matches the existing cleaned_comment column in
youtube_comments_for_labelling.csv, with the addition of encoding
normalisation and explicit stopword/stemming steps for consistency
with the Interim Report's stated methodology (C2).

Outputs
-------
02_processed_data/
    youtube_labelled_processed.csv   - 300 labelled YouTube rows
    reddit_labelled_processed.csv    - 297 labelled Reddit rows
    combined_labelled_processed.csv  - merged for WEKA training
    youtube_full_processed.csv       - full YouTube corpus (2619 rows)
    reddit_full_processed.csv        - full Reddit corpus (~15k rows)
```

```
04_weka_files/
    training_data.arff              - combined labelled set for WEKA
    youtube_full.arff               - full YouTube corpus for WEKA
    reddit_full.arff                - full Reddit corpus for WEKA
"""

from __future__ import annotations

import re
import pandas as pd
from pathlib import Path

import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Download required NLTK data if not already present
nltk.download('stopwords', quiet=True)
nltk.download('punkt', quiet=True)

# ------------------------------------------------------------------------
---
# Paths
# ------------------------------------------------------------------------
---

BASE = Path(r"C:\Users\lenovo\kpop-sentiment-analysis")

YOUTUBE_LABELLED = BASE / "03_labeled_data" /
"youtube_comments_for_labelling.csv"
REDDIT_LABELLED  = BASE / "03_labeled_data" /
"reddit_labelling_template.csv"
YOUTUBE_FULL     = BASE / "01_raw_data" / "youtube" /
"youtube_comments_en.csv"
REDDIT_EN_DIR    = BASE / "01_raw_data" / "reddit" / "cleaned_en"

OUT_PROCESSED    = BASE / "02_processed_data"
OUT_WEKA         = BASE / "04_weka_files"
OUT_PROCESSED.mkdir(parents=True, exist_ok=True)
OUT_WEKA.mkdir(parents=True, exist_ok=True)

# ------------------------------------------------------------------------
---
# Cleaning pipeline
# ------------------------------------------------------------------------
---

STOP_WORDS = set(stopwords.words('english'))
STEMMER    = PorterStemmer()

# Encoding fix map for common UTF-8 mojibake sequences from Pushshift
ENCODING_FIXES = {
    'â€™': "'",
```

```
    'â€œ': '"',
    'â€':  '"',
    'â€"': '-',
    'â€"': '-',
    'Â':   '',
    'â€¦': '...',
}


def fix_encoding(text: str) -> str:
    """Fix common UTF-8 mojibake sequences from Pushshift dumps."""
    for bad, good in ENCODING_FIXES.items():
        text = text.replace(bad, good)
    return text


def clean(text: str) -> str:
    """Apply the full cleaning pipeline to a single comment."""
    if not isinstance(text, str) or not text.strip():
        return ''

    # Step 1: encoding fix
    text = fix_encoding(text)

    # Step 2: lowercase
    text = text.lower()

    # Step 3: remove URLs
    text = re.sub(r'http\S+|www\S+', '', text)

    # Step 4: remove non-ASCII (emoji, Hangul residue, etc.)
    text = text.encode('ascii', errors='ignore').decode('ascii')

    # Step 5: remove digits
    text = re.sub(r'\d+', '', text)

    # Step 6: remove punctuation (keep spaces)
    text = re.sub(r'[^\w\s]', '', text)

    # Step 7: tokenise
    tokens = text.split()

    # Step 8: stopword removal
    tokens = [t for t in tokens if t not in STOP_WORDS]

    # Step 9: stemming
    tokens = [STEMMER.stem(t) for t in tokens]

    # Step 10: rejoin
    return ' '.join(tokens)


# ------------------------------------------------------------------------
---
```

```
# ARFF export
# ------------------------------------------------------------------------
---

def to_arff(df: pd.DataFrame,
            text_col: str,
            label_col: str,
            relation_name: str,
            path: Path,
            label_values: list[str] | None = None) -> None:
    """
    Write a simple bag-of-words ARFF file suitable for WEKA
StringToWordVector.

    The text column is written as a STRING attribute so that WEKA's
    StringToWordVector filter can convert it to TF-IDF features before
    classification. This is the standard approach for text classification
    in WEKA when you don't want to pre-compute the vocabulary yourself.
    """
    if label_values is None:
        label_values = sorted(df[label_col].dropna().unique().tolist())

    lines = [
        f"@relation {relation_name}",
        "",
        "@attribute text STRING",
        f"@attribute class {{{','.join(label_values)}}}",
        "",
        "@data",
    ]

    for _, row in df.iterrows():
        text  = str(row[text_col]).replace("'", "\\'").replace('"', '\\"')
        label = str(row[label_col])
        # ARFF string values are wrapped in single quotes
        lines.append(f"'{text}',{label}")

    path.write_text('\n'.join(lines), encoding='utf-8')
    print(f"  -> ARFF: {path}  ({len(df)} instances)")


# ------------------------------------------------------------------------
---
# Processing functions
# ------------------------------------------------------------------------
---

def process_youtube_labelled() -> pd.DataFrame:
    """Load, filter to labelled rows, apply cleaning, return DataFrame.

    Note: the existing 'cleaned_comment' column in the source file used a
    lighter pipeline (no stopword removal, no stemming). This function
    re-cleans from the raw 'comment' column using the full pipeline
defined
```

```
    above, which matches the methodology stated in the Interim Progress
    Report (C2: tokenisation, stopword removal, stemming via NLTK).
    The 'cleaned_comment' column is intentionally ignored.
    """
    df = pd.read_csv(YOUTUBE_LABELLED)
    # Keep only labelled rows
    df = df[df['sentiment'].notna()].copy()
    df = df.rename(columns={'sentiment': 'label', 'comment': 'raw_text',
                            'group_comeback': 'comeback'})
    # Re-clean from raw text (not cleaned_comment)
    df['cleaned_text'] = df['raw_text'].apply(clean)
    df['source'] = 'youtube'
    return df[['comeback', 'raw_text', 'cleaned_text', 'label', 'source']]


def process_reddit_labelled() -> pd.DataFrame:
    """Load labelled Reddit rows, apply cleaning, return DataFrame."""
    df = pd.read_csv(REDDIT_LABELLED)
    df = df[df['label'].notna()].copy()
    df = df.rename(columns={'text': 'raw_text'})
    df['cleaned_text'] = df['raw_text'].apply(clean)
    df['source'] = 'reddit'
    return df[['comeback', 'raw_text', 'cleaned_text', 'label', 'source']]


def process_youtube_full() -> pd.DataFrame:
    """Load the full English-filtered YouTube corpus and clean it."""
    df = pd.read_csv(YOUTUBE_FULL)
    df = df.rename(columns={'comment': 'raw_text',
                            'group_comeback': 'comeback'})
    df['cleaned_text'] = df['raw_text'].apply(clean)
    df['label'] = '?'   # unknown — to be predicted by classifier
    df['source'] = 'youtube'
    return df[['comeback', 'raw_text', 'cleaned_text', 'label', 'source']]


def process_reddit_full() -> pd.DataFrame:
    """Load all six English-filtered Reddit CSVs and clean them."""
    frames = []
    for path in sorted(REDDIT_EN_DIR.glob("*_en.csv")):
        sub = pd.read_csv(path)
        sub = sub.rename(columns={'text': 'raw_text'})
        frames.append(sub)
    df = pd.concat(frames, ignore_index=True)
    df['cleaned_text'] = df['raw_text'].apply(clean)
    df['label'] = '?'
    df['source'] = 'reddit'
    return df[['comeback', 'raw_text', 'cleaned_text', 'label', 'source']]


# ------------------------------------------------------------------------
---
# Main
```

```
# ----------------------------------------------------------------------
---

def main() -> None:
    label_values = ['negative', 'neutral', 'positive']

    print("Processing YouTube labelled ...")
    yt_lab = process_youtube_labelled()
    yt_lab.to_csv(OUT_PROCESSED / "youtube_labelled_processed.csv",
index=False)
    print(f"  {len(yt_lab)} rows  |
{yt_lab['label'].value_counts().to_dict()}")

    print("Processing Reddit labelled ...")
    rd_lab = process_reddit_labelled()
    rd_lab.to_csv(OUT_PROCESSED / "reddit_labelled_processed.csv",
index=False)
    print(f"  {len(rd_lab)} rows  |
{rd_lab['label'].value_counts().to_dict()}")

    print("Combining labelled sets ...")
    combined = pd.concat([yt_lab, rd_lab], ignore_index=True)
    combined.to_csv(OUT_PROCESSED / "combined_labelled_processed.csv",
index=False)
    print(f"  {len(combined)} rows  |
{combined['label'].value_counts().to_dict()}")

    print("Processing full YouTube corpus ...")
    yt_full = process_youtube_full()
    yt_full.to_csv(OUT_PROCESSED / "youtube_full_processed.csv",
index=False)
    print(f"  {len(yt_full)} rows")

    print("Processing full Reddit corpus ...")
    rd_full = process_reddit_full()
    rd_full.to_csv(OUT_PROCESSED / "reddit_full_processed.csv",
index=False)
    print(f"  {len(rd_full)} rows")

    print("\nExporting ARFF files ...")
    to_arff(combined, 'cleaned_text', 'label',
            'kpop_sentiment_training', OUT_WEKA / 'training_data.arff',
            label_values)
    to_arff(yt_full, 'cleaned_text', 'label',
            'youtube_full', OUT_WEKA / 'youtube_full.arff',
            label_values)
    to_arff(rd_full, 'cleaned_text', 'label',
            'reddit_full', OUT_WEKA / 'reddit_full.arff',
            label_values)

    print("\nDone. Summary:")
    print(f"  Training instances : {len(combined)}")
    print(f"  YouTube to predict : {len(yt_full)}")
    print(f"  Reddit to predict  : {len(rd_full)}")
```

```
    print(f"  ARFF files in      : {OUT_WEKA}")


if __name__ == "__main__":
    main()
```

Script 5: sample_for_labelling.py

```
"""
sample_for_labelling.py
-----------------------
Draws a stratified random sample of 50 comments per comeback from both
Reddit and YouTube English-filtered corpora, producing two labelling
workbooks. Adds an empty 'label' column for the user to fill with
positive, negative, or neutral.

Rationale
    Random stratified sampling is reproducible (fixed seed) and avoids
    self-selection bias that would occur if the labeller chose which
    comments to label. This is the standard practice for building a
    labelled training set from a larger corpus.

Output
    03_labeled_data/reddit_labelling_template.csv
    03_labeled_data/youtube_labelling_template.csv
"""

from __future__ import annotations

import pandas as pd
from pathlib import Path


RANDOM_SEED        = 42
SAMPLE_PER_COMEBACK = 50

REDDIT_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit\cleaned_en")
YT_FILE    = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\youtube\youtube_comments_en.csv")
OUT_DIR    = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\03_labeled_data")
OUT_DIR.mkdir(parents=True, exist_ok=True)


def sample_reddit() -> pd.DataFrame:
    frames = []
    for path in sorted(REDDIT_DIR.glob("*_en.csv")):
        df = pd.read_csv(path)
        if len(df) < SAMPLE_PER_COMEBACK:
            print(f"  WARN: {path.name} has only {len(df)} rows, taking
all")
            sampled = df.copy()
```

```
        else:
            sampled = df.sample(n=SAMPLE_PER_COMEBACK,
random_state=RANDOM_SEED)
        frames.append(sampled)
    result = pd.concat(frames, ignore_index=True)
    result["label"] = ""   # empty column for manual labelling
    result["source"] = "reddit"
    return result


def sample_youtube() -> pd.DataFrame:
    df = pd.read_csv(YT_FILE)
    frames = []
    for cb, sub in df.groupby("group_comeback"):
        if len(sub) < SAMPLE_PER_COMEBACK:
            print(f"  WARN: {cb} has only {len(sub)} rows, taking all")
            sampled = sub.copy()
        else:
            sampled = sub.sample(n=SAMPLE_PER_COMEBACK,
random_state=RANDOM_SEED)
        frames.append(sampled)
    result = pd.concat(frames, ignore_index=True)
    result["label"] = ""
    result["source"] = "youtube"
    return result


def main() -> None:
    print("Sampling Reddit ...")
    rd = sample_reddit()
    rd_out = OUT_DIR / "reddit_labelling_template.csv"
    rd.to_csv(rd_out, index=False)
    print(f"  -> {rd_out}  ({len(rd)} rows)")

    print("Sampling YouTube ...")
    yt = sample_youtube()
    yt_out = OUT_DIR / "youtube_labelling_template.csv"
    yt.to_csv(yt_out, index=False)
    print(f"  -> {yt_out}  ({len(yt)} rows)")


if __name__ == "__main__":
    main()
```

Script 6: collect_youtube_metrics.py

```
"""
collect_youtube_metrics.py
--------------------------
Collects video-level engagement metrics for the six comeback MVs
using the YouTube Data API v3.

Saves to: 01_raw_data/youtube/youtube_metrics.csv
```

```
"""

import requests
import pandas as pd
from pathlib import Path

# Your YouTube Data API v3 key
# Replace with your actual key from credentials folder
API_KEY = "YOUR_YOUTUBE_API_KEY_HERE"

# Video IDs identified from the comments corpus
VIDEOS = {
    "aespa_Whiplash":           "jWQx2f-CErU",
    "ATEEZ_IceOnMyTeeth":       "5OflOlcHLb8",
    "NCTDREAM_WhenImWithYou":   "B1qq8IvzSz4",
    "StrayKids_ChkChkBoom":     "0P0aQreFs8w",
    "TWICE_Strategy":           "Sz_wWzgh-vQ",
    "IVE_RebelHeart":           "g36q0ZLvygQ",
}

OUTPUT = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\youtube\youtube_metrics.csv")


def get_video_metrics(video_id: str, api_key: str) -> dict:
    url = "https://www.googleapis.com/youtube/v3/videos"
    params = {
        "part": "statistics,snippet",
        "id": video_id,
        "key": api_key,
    }
    response = requests.get(url, params=params)
    data = response.json()

    if not data.get("items"):
        print(f"  WARNING: No data returned for {video_id}")
        return {}

    item = data["items"][0]
    stats = item["statistics"]

    return {
        "video_id":      video_id,
        "view_count":    int(stats.get("viewCount", 0)),
        "like_count":    int(stats.get("likeCount", 0)),
        "comment_count": int(stats.get("commentCount", 0)),
        "title":         item["snippet"]["title"],
        "published_at":  item["snippet"]["publishedAt"],
    }


def main():
    rows = []
    for comeback, video_id in VIDEOS.items():
```

```
        print(f"Fetching metrics for {comeback} ({video_id}) ...")
        metrics = get_video_metrics(video_id, API_KEY)
        if metrics:
            metrics["comeback"] = comeback
            # Compute like-to-view ratio
            if metrics["view_count"] > 0:
                metrics["like_to_view_ratio"] = metrics["like_count"] /
metrics["view_count"]
            else:
                metrics["like_to_view_ratio"] = 0.0
            rows.append(metrics)
            print(f"  Views: {metrics['view_count']:,}  Likes:
{metrics['like_count']:,}  Comments: {metrics['comment_count']:,}")

    df = pd.DataFrame(rows)
    cols = ["comeback", "video_id", "title", "published_at",
            "view_count", "like_count", "comment_count",
"like_to_view_ratio"]
    df = df[cols]

    OUTPUT.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(OUTPUT, index=False)
    print(f"\nSaved to {OUTPUT}")
    print(df[["comeback", "view_count", "like_count", "comment_count",
"like_to_view_ratio"]].to_string(index=False))


if __name__ == "__main__":
    main()
```

Appendix C: Tableau Dashboard Screenshot

APPENDIX C: TABLEAU DASHBOARD SCREENSHOT

Introduction

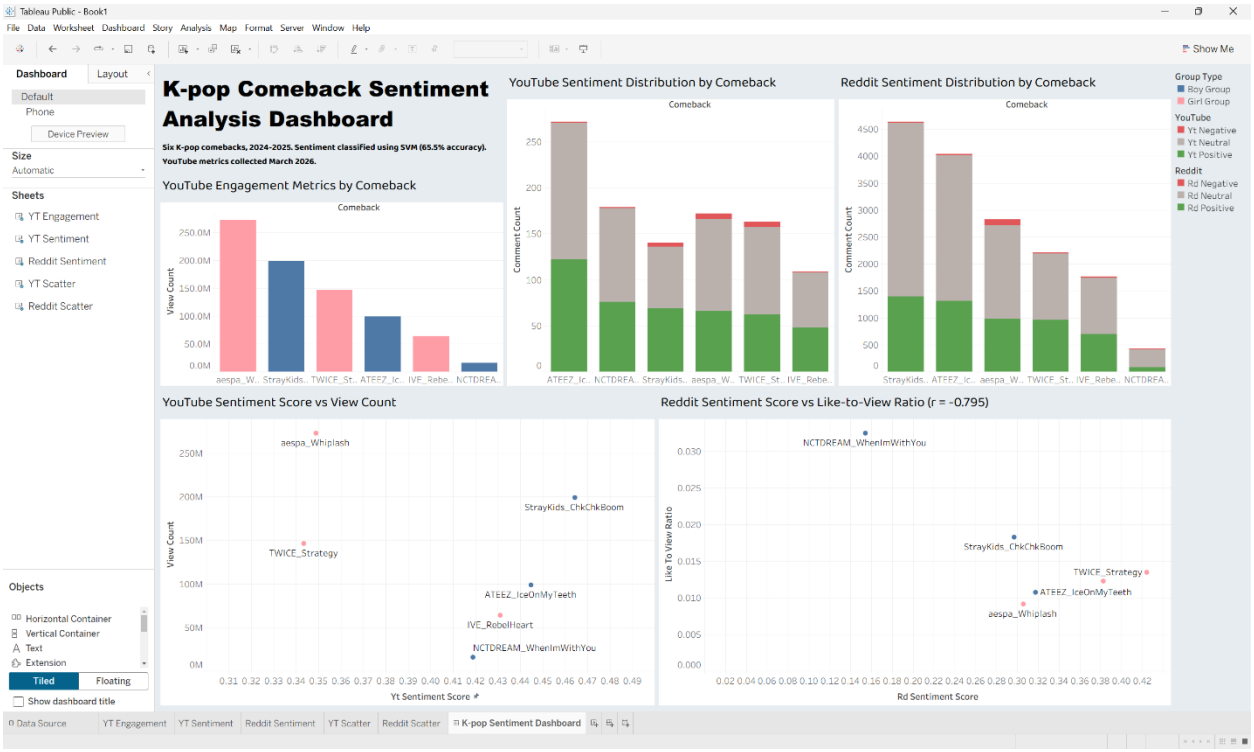
This appendix contains screenshots of the K-pop Comeback Sentiment Analysis Dashboard. The dashboard is available online at: K-pop Sentiment Dashboard

The full set of individual chart screenshots is available in the project GitHub repository (see 09_reports/screenshots/).

Content

Tableau Dashboard

SCREENSHOT 1 - TABLEAU DASHBOARD



Appendix D: WEKA Process Screenshots

Appendix D: WEKA Process Screenshots

Introduction

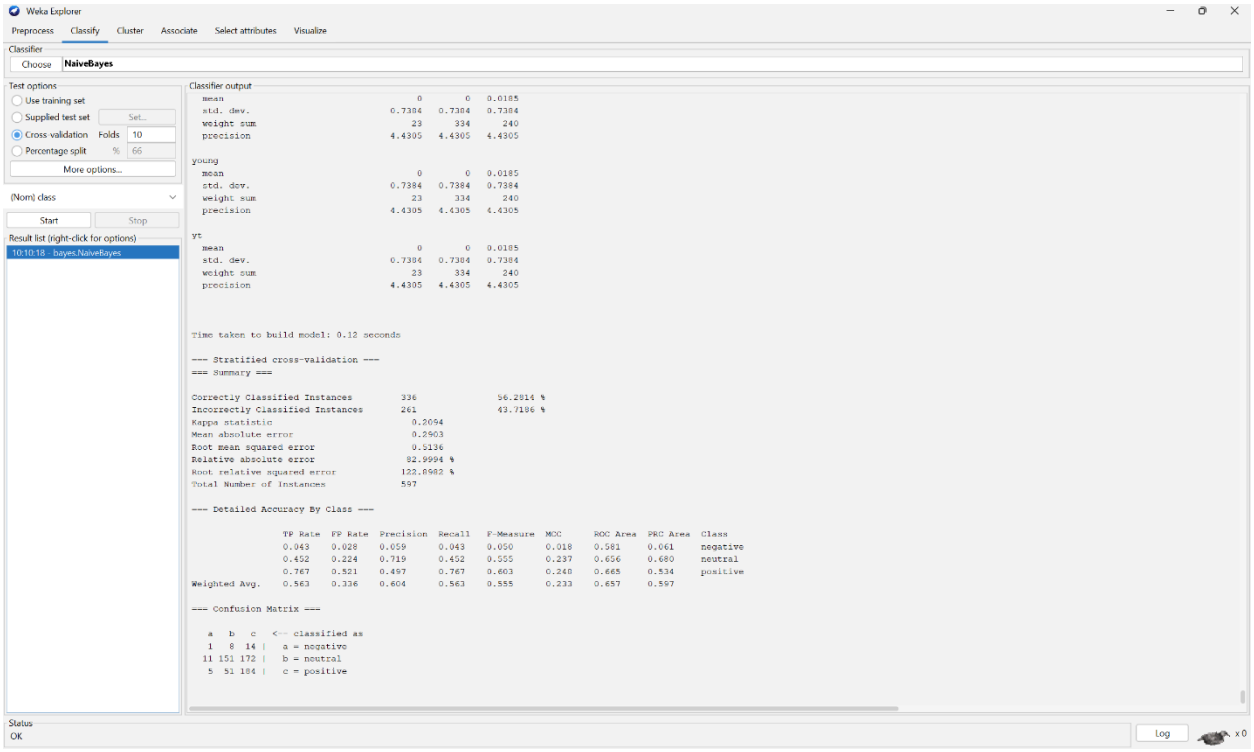
This appendix documents the classification workflow in WEKA through screenshots captured at key stages of model configuration, training, and evaluation.

Additional WEKA process screenshots are available in the project GitHub repository (see 09_reports/screenshots/).

Content

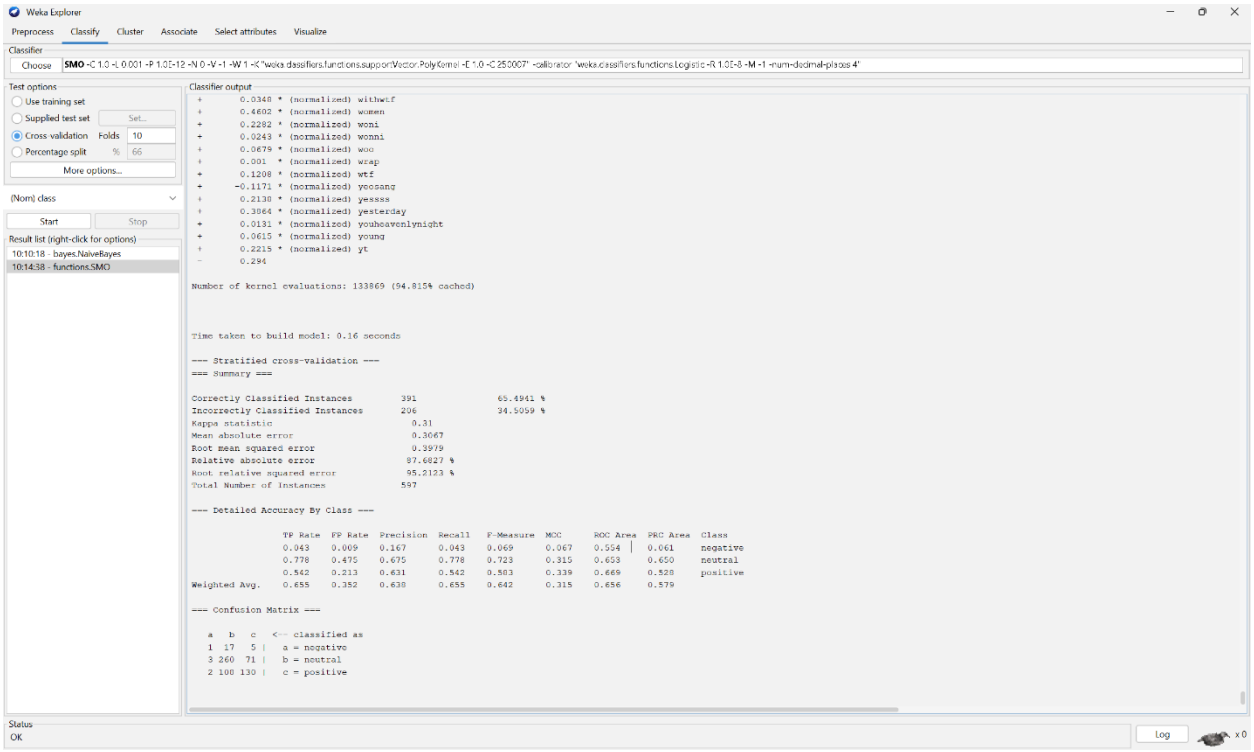
Naïve Bayes Results WEKA

Screenshot 2 - Naïve Bayes Results WEKA



SVM (SMO) Results WEKA

SCREENSHOT 3 - SVM RESULTS WEKA



Appendix E: Updated Risk Register

APPENDIX E: UPDATE RISK REGISTER

Introduction

This appendix presents the project risk register, updated to reflect risks encountered during development and the mitigation strategies applied.

Content

Risk Register

TABLE E - RISK REGISTER

ID	Description	Likelihood	Impact	Mitigation	Status
R1	Twitter API access not granted	Occurred	High	Replaced with YouTube comments and Reddit Pushshift data	Resolved
R2	Reddit API access not granted	Occurred	High	Used Pushshift archive dumps instead	Resolved
R3	API key exposed on GitHub	Occurred	High	Key revoked, new restricted key created, git history cleaned	Resolved
R4	Data collection script lost	Occurred	Medium	Script reconstructed from existing output files	Resolved
R5	Incorrect IVE collection window	Occurred	Medium	Re-collected using correct January 2025 window	Resolved
R6	WEKA GUI prediction export failure	Occurred	Medium	Pipeline replicated in scikit-learn LinearSVC	Resolved
R7	Class imbalance in training data	Occurred	Medium	Acknowledged as limitation; class weighting applied	Partially mitigated
R8	Large files committed to Git	Occurred	Low	Removed via rebase; .gitignore updated	Resolved

Appendix F: AI Use Detail

Introduction

This appendix provides a detailed account of AI tool usage throughout the project, as referenced in the Introduction chapter.

Content

AI Use Details

TABLE F - AI USE DETAILS

Aspect	Description
Tools used	Claude (Anthropic, via claude.ai); Grammarly (AI-powered writing assistant); Google Scholar Labs (AI-assisted search feature)
Phases	Literature search, development support, and proofreading
Specific uses	Identifying relevant academic sources and exploring research topics (Google Scholar Labs), debugging Python scripts, clarifying technical concepts, resolving development issues, proofreading for grammar and sentence clarity
Not used for	Generating report content, developing methodology, conducting analysis, or producing final code without review
Oversight	All sources identified through AI-assisted search were independently verified and manually reviewed; all AI outputs were critically evaluated before use; all ideas, analysis, and technical work are my own